

LLM Inference Systems

Large-language-model inference is the systems problem of producing tokens quickly, cheaply, and predictably from a trained model. The model architecture matters, but production performance is usually determined by memory movement, scheduling, caching, numerical formats, and communication between GPUs.

This guide develops twelve core ideas from first principles and then turns them into interview-ready explanations. Unless stated otherwise, assume an autoregressive Transformer in which **prefill** processes the input prompt in parallel and **decode** generates one new token per sequence per iteration.

KV Cache

1. Overview

Definition. The key-value (KV) cache stores the attention keys and values produced for earlier tokens so that decoding a new token does not recompute them.

In a causal Transformer, token t attends to tokens $0..t$. The earlier keys and values do not change during ordinary autoregressive decoding. Saving them turns repeated work into reads. KV caching is used by nearly every production decoder-only LLM server, from a single-GPU chatbot to a distributed inference cluster.

It matters because it greatly reduces decode computation, but it also consumes large amounts of GPU memory and creates a bandwidth-heavy workload. Interviewers ask about it to test whether a candidate can connect Transformer math to GPU memory capacity, latency, batching, and serving-system design.

2. Core Idea

For one attention layer:

$$\begin{aligned} Q &= XW_q & K &= XW_k & V &= XW_v \\ \text{Attention}(Q, K, V) &= \text{softmax}(QK^T / \sqrt{d})V \end{aligned}$$

During decode, only the newest token's hidden state is new. Its Q , K , and V are computed, but the K and V for previous tokens are identical to those computed earlier.

Analogy. A student answering question 20 can keep notes from questions 1–19. Rewriting all previous notes before every answer is correct but wasteful; retaining them requires storage but saves repeated work.

Small example:

```
Prompt: A B C
Prefill cache: K[A,B,C], V[A,B,C]

Generate D:
```

```
compute Q[D], K[D], V[D]
attend Q[D] to cached K[A,B,C] plus K[D]
append K[D], V[D]
```

Generate E:

```
compute only Q[E], K[E], V[E]
attend to cached K[A,B,C,D] plus K[E]
```

Step by step:

1. Prefill computes attention for all prompt tokens and stores every layer's K/V tensors.
2. Decode computes projections for the latest token.
3. The attention kernel reads the sequence's cached keys and values.
4. The new K/V vectors are appended to the cache.
5. The cache grows until generation stops or the context window is truncated.

A common size estimate for standard multi-head attention is:

```
KV bytes per sequence
= 2 × layers × cached_tokens × kv_heads × head_dim × bytes_per_element
```

The factor 2 is for K and V. For 32 layers, 32 KV heads, head dimension 128, 4,096 tokens, and FP16:

```
2 × 32 × 4096 × 32 × 128 × 2 bytes = 2 GiB
```

This explains why an apparently small number of long requests can exhaust a GPU even when model weights fit.

3. Important Subtopics

Prefill versus decode

Prefill processes many prompt tokens at once and is often compute-intensive. Decode processes one token per active sequence and repeatedly scans the growing cache, so it is often memory-bandwidth-bound.

Interview angle: optimizing one phase does not automatically optimize the other.

Multi-head, multi-query, and grouped-query attention

In multi-head attention (MHA), every query head has distinct K/V heads. Multi-query attention (MQA) shares one K/V head across all query heads. Grouped-query attention (GQA) uses fewer K/V heads than query heads. MQA/GQA shrink cache capacity and bandwidth costs, often with a modest quality trade-off. Interviewers expect the cache formula to use `kv_heads`, not blindly use `attention_heads`.

Cache layout

The physical order of layer, sequence, head, token, and head-dimension axes affects coalescing and kernel efficiency. A layout good for appending one token may differ from one good for reading all tokens. Serving engines use layouts tailored to their attention kernels.

Contiguous versus paged allocation

A contiguous cache reserves one large region per sequence and is simple, but variable lengths cause over-reservation and fragmentation. Paged KV caches allocate fixed-size blocks as tokens arrive. This is the storage foundation of paged attention.

Cache precision

K/V values are commonly FP16 or BF16; FP8 or INT8 caches reduce memory traffic and capacity but require scales and can affect accuracy. This is distinct from quantizing model weights.

Sliding-window and sink-token caches

Models with local attention may retain only a recent window, sometimes plus a few initial “attention sink” tokens. This bounds memory growth, but only when the model and positional scheme support it.

4. Real-World Example

Consider a chat service running a 32-layer GQA model. A user sends a 2,000-token conversation and asks for 200 new tokens. Prefill creates K/V entries for 2,000 tokens. Each decode step reads those entries, appends one entry per layer, and samples a token. If the user sends another turn and the server preserves the session cache, it can reuse the shared conversation prefix rather than prefill the entire history again. The scheduler must account for both weight memory and the cache blocks required by all concurrent users.

5. Diagrams / Mental Models

Without cache	With cache
step 1: compute [A B C D]	prefill [A B C]
step 2: compute [A B C D E]	decode D: read [A B C] + append D
step 3: compute [A B C D E F]	decode E: read [A B C D] + append E
repeats old K/V projections	computes only new-token K/V

Resource	Prefill	Decode with KV cache
Tokens processed per sequence/iteration	Many	One
Parallelism	High	Limited per sequence
Attention data	Create cache	Read old cache, append new entry
Typical bottleneck	Compute or memory	Often memory bandwidth/capacity

6. Common Interview Questions

#	Question and answer	Interviewer expects	Common mistake
1	What is a KV cache? Stored K/V tensors for previous tokens, reused during causal decode.	Explain what is cached and why it is invariant.	Saying outputs or queries are cached.
2	Why not cache queries? Old queries are not needed to compute the new token; the new query attends to old keys/values.	Direction of causal attention.	Claiming Q is too small rather than unnecessary.
3	What does it save? Recomputing old K/V projections and old-token attention states during each decode step.	Compute-for-memory trade-off.	Claiming attention becomes constant-time; it still reads more tokens as context grows.
4	How does cache size scale? Linearly with layers, tokens, KV heads, head dimension, element size, and concurrent sequences.	State or derive the formula.	Using query-head count for GQA/MQA.
5	Why can decode be bandwidth-bound? Each step performs little arithmetic per byte while reading a large cache and weights.	Arithmetic intensity argument.	Blaming only kernel launch overhead.
6	MHA vs GQA vs MQA for caching? GQA/MQA use fewer KV heads and therefore less cache memory/bandwidth.	Capacity-performance-quality trade-off.	Saying they reduce query heads.
7	When is the cache created? During prefill for prompt tokens, then extended once per generated token.	Two-phase lifecycle.	Saying it exists in training in the same way.
8	Can requests share a KV cache? Only identical token prefixes with compatible model/configuration can safely share immutable prefix blocks.	Exact token identity and copy-on-write.	Sharing semantically similar text.
9	How is cache memory reclaimed? Free a sequence's blocks on completion/cancellation; reusable allocators avoid expensive device allocations.	Lifecycle and allocator design.	Waiting for a global GPU reset.
10	Does cache quantization equal weight quantization? No. They target different tensors, access patterns, error sensitivity, and memory costs.	Separate policies and scales.	Treating "INT8 model" as proof the cache is INT8.

7. Deep-Dive Questions

1. **Why is decode attention still $O(L)$ per new token with a cache?** The new query must compare against L cached keys and combine L values. The cache eliminates recomputation of old projections

and old states, not the scan over context. Generating T tokens therefore still has a growing attention cost.

2. **How does tensor parallelism affect the cache?** Attention/KV heads are usually sharded across tensor-parallel ranks. Each rank stores its local heads; GQA configurations may require replication or careful head-to-rank mapping when KV heads are fewer than ranks.
3. **Why is beam search expensive for cache memory?** Beams share the same prefix but diverge later. A naive implementation copies the full cache per beam; block-level sharing with copy-on-write stores the prefix once and copies only divergent blocks.
4. **What invalidates a reusable cache?** Different token IDs, model weights/adapters, positional encoding state, attention settings, or cache representation. Even visually identical text can tokenize differently.
5. **How would you choose KV block size?** Balance metadata and kernel overhead from small blocks against internal fragmentation and copy-on-write waste from large blocks. Measure realistic sequence-length distributions and kernel behavior.

8. Comparison Tables

Approach	Decode compute	Memory use	Main limitation
Recompute full prefix	Very high	Low persistent cache	Latency grows badly
Contiguous KV cache	Low	High; may over-reserve	Fragmentation and resizing
Paged KV cache	Low	High but allocated on demand	Indirection and block metadata
Quantized KV cache	Low plus dequantization	Lower	Accuracy and kernel support
Sliding-window cache	Low	Bounded	Cannot represent unrestricted full attention

9. Common Mistakes

- Confusing the KV cache with a cache of generated tokens; token IDs are tiny compared with per-layer K/V tensors.
- Assuming caching makes attention $O(1)$.
- Forgetting the cache exists independently at every Transformer layer.
- Estimating capacity with total attention heads instead of KV heads for GQA/MQA.
- Ignoring cache memory when calculating maximum batch size.
- Assuming a cache can be reused after changing the model, LoRA adapter, or tokenization.
- Optimizing cache capacity while ignoring the memory bandwidth required to read it.

10. Edge Cases / Special Cases

- Empty prompts still need a beginning token or model-specific initialization.
- Cancellation must free blocks even if a request ends between decode iterations.
- Beam search and parallel sampling need prefix sharing or can multiply memory usage.

- Sliding-window models may evict old tokens, but absolute/rotary positional behavior must remain correct.
- CUDA Graph capture often requires stable addresses or indirection tables, complicating a dynamic allocator.
- Very short prompts may not benefit enough to justify elaborate prefix reuse bookkeeping.
- Chunked prefill interleaves cache creation for long prompts with ongoing decodes.

11. How to Explain in Interview

“A KV cache stores every layer's keys and values for previous tokens. During autoregressive decode, old K/V tensors do not change, so each step computes K/V only for the new token, reads the cached context, and appends one entry. This cuts repeated computation but makes memory capacity and bandwidth major serving bottlenecks; GQA, paging, and cache quantization address those costs.”

12. Quick Revision Notes

- **Definition:** per-layer cached keys and values for prior tokens.
- **Size:** $2 \times \text{layers} \times \text{tokens} \times \text{kv_heads} \times \text{head_dim} \times \text{bytes}$ per sequence.
- **Lifecycle:** create in prefill, append in decode, free at completion.
- **Key comparison:** MHA has the largest cache; GQA/MQA reduce KV heads.
- **Must remember:** cache saves recomputation, not the need to attend over context.
- **Trap:** cache memory, not only weights, often limits concurrency.

13. Practice Tasks

1. Calculate cache size for a 40-layer model with 8 KV heads, head dimension 128, BF16, and 8,192 cached tokens.
2. Write a small Python attention decoder that compares full-prefix recomputation with cached K/V and checks equal outputs.
3. Plot cache capacity versus context length for MHA, GQA, and MQA.
4. Design a block allocator supporting allocate, append, fork, and free for beam search.
5. Profile prefill and decode separately and explain their arithmetic intensity.

14. Final Cheat Sheet

Item	Remember
Core definition	Reuse old per-layer K/V tensors during causal decoding
Why it matters	Much less recomputation; much more persistent memory
Most asked	Size formula, why Q is not cached, MHA vs GQA/MQA, bandwidth bottleneck
Comparisons	Contiguous vs paged; FP16 vs quantized; full vs sliding-window
One-line answer	“KV caching trades GPU memory for faster autoregressive decoding by reusing invariant past keys and values.”

Continuous Batching

1. Overview

Definition. Continuous batching, also called iteration-level or in-flight batching, rebuilds the active batch at token-generation boundaries so completed requests can leave and waiting requests can join without waiting for the whole original batch.

It matters because LLM requests have different prompt and output lengths. In a static batch, a short request leaves an empty slot until the longest request finishes. Continuous batching keeps the GPU populated, increasing throughput and usually reducing queueing time. It is central to production LLM servers such as vLLM-style engines, TensorRT-LLM-based services, and custom inference schedulers.

Interviewers ask about it because it combines GPU utilization, request scheduling, latency-throughput trade-offs, memory admission control, and the special one-token-at-a-time nature of autoregressive decode.

2. Core Idea

Analogy. A restaurant with one fixed seating waits until every diner at a table leaves before seating anyone new. A continuously batched restaurant fills each seat as soon as its diner leaves. The kitchen still cooks groups of dishes together, but group membership changes over time.

Example with maximum batch size three:

```
iteration 1: [A, B, C] -> one token each
iteration 2: [A, B, C] -> B finishes
iteration 3: [A, D, C] -> D joins immediately
iteration 4: [A, D, C] -> A finishes
iteration 5: [E, D, C] -> E joins immediately
```

Step by step:

1. Requests enter a waiting queue.
2. The scheduler checks token budget, KV-cache blocks, priorities, and deadlines.
3. It forms one model iteration from active decode sequences and possibly prompt chunks.
4. The GPU runs the batched forward pass.
5. Sampling produces one or more tokens and identifies stopped/cancelled requests.
6. Finished requests release cache memory; admitted requests take their places.
7. The cycle repeats until the queue is empty.

The scheduling unit is usually a **token budget**, not merely number of requests, because a 4,000-token prefill costs far more than one decode token.

3. Important Subtopics

Iteration-level scheduling

Membership can change after each decode step. This is the defining difference from static/request-level batching. It improves occupancy but adds scheduling and metadata overhead.

Prefill and decode interference

Prefill involves many tokens and can monopolize the GPU; decode needs frequent small iterations for low inter-token latency. Chunked prefill divides long prompts into bounded pieces so they can coexist with decodes. Interview angle: throughput gains can damage token latency if prefill is not controlled.

Admission control

A request needs enough KV-cache capacity now and as it grows. An engine may reserve worst-case space, allocate on demand, reject early, preempt another request, or swap/recompute cache. Correct admission prevents out-of-memory failures.

Scheduling policies

First-come-first-served is simple. Shortest-job-first can reduce mean latency but needs length estimates and risks starvation. Priority and deadline policies support service tiers. Interviewers expect fairness to be discussed alongside utilization.

Preemption

When a high-priority request arrives or cache is exhausted, a sequence may be paused. Its cache can stay resident, be swapped to host memory, or be discarded and recomputed. Each option trades memory, transfer cost, and compute.

Ragged batches

Sequences have different context lengths. Kernels need length arrays and block tables rather than a single rectangular tensor padded to the maximum. Paged attention makes these variable-length batches efficient.

4. Real-World Example

A backend receives interactive chat traffic plus long summarization jobs. Static batching groups eight requests, but seven chats finish after 40 tokens while one summary generates 1,000; most slots remain idle. Continuous batching removes each completed chat and admits another. A token-budget scheduler chunks long prefills and prioritizes already-decoding chats, improving GPU utilization without allowing one huge document to create a long pause between chat tokens.

5. Diagrams / Mental Models

Static batching

time ->	1	2	3	4	5	6	7	8	
A	x	x	x	.	.	.	.	.	
B	x	x	x	x	x	x	x	x	batch ends here
C	x	x	.	.	.	.	.	.	

Continuous batching

time ->	1	2	3	4	5	6	7	8
slot 1	A	A	A	D	D	E	E	E
slot 2	B	B	B	B	B	B	B	B
slot 3	C	C	F	F	G	G	G	H

Metric	Meaning
Time to first token (TTFT)	Arrival to first generated token; queueing and prefill dominate
Time per output token (TPOT)	Average decode spacing after first token
Inter-token latency (ITL)	Gap between individual streamed tokens, including jitter
Throughput	Tokens or requests completed per second

6. Common Interview Questions

#	Question and answer	Interviewer expects	Common mistake
1	What is continuous batching? Changing batch membership at inference-iteration boundaries.	Contrast with static batching.	Describing ordinary dynamic request collection only.
2	Why is it effective for LLMs? Output lengths vary and decoding naturally has repeated token boundaries.	Variable service times plus iterative decode.	Saying all sequences take equal compute.
3	How does it improve utilization? Finished slots are filled immediately rather than idling until the longest request ends.	Timeline explanation.	Claiming GPU occupancy is always 100%.
4	What is its main cost? Scheduler/metadata complexity, ragged kernels, and potential latency interference.	Trade-offs, not "free speed."	Mentioning only CPU scheduling overhead.
5	How do prefill and decode interact? Large prefills improve throughput but can delay decode iterations; chunking and budgets control this.	TTFT versus TPOT.	Treating every token as equal cost.
6	Batch size or token budget? Token budget better reflects work because request lengths and phases differ.	Per-iteration scheduled-token count.	Using request count alone.
7	What happens when KV memory is full? Stop admission, preempt/swap/recompute, or reject according to policy.	Memory-aware scheduling.	Admitting and hoping allocations succeed.
8	Can continuous batching hurt latency? Yes; larger iterations and prefills can increase ITL even as throughput rises.	Tail latency and jitter.	Assuming throughput and latency always improve together.
9	How is fairness maintained? Age/priority-aware queues, bounded prompt chunks, quotas, or anti-starvation rules.	Policy and starvation.	Pure shortest-job-first without caveat.
10	When does it help less? Low traffic, homogeneous short jobs, CPU-bound serving, or kernels too small	Workload-dependent answer.	Saying it universally improves every

#	Question and answer	Interviewer expects	Common mistake
	to batch efficiently.		deployment.

7. Deep-Dive Questions

1. **How would you optimize both TTFT and TPOT?** Limit queue delay, chunk long prefills, reserve regular decode opportunities, and tune token budgets from an SLO rather than maximum throughput alone. Track p95/p99, not only averages.
2. **Why can a larger batch reduce throughput in some cases?** It may exceed efficient kernel shapes, increase memory pressure, cause cache thrashing, or lengthen each iteration enough to hurt admission and tail behavior. Benchmark batch/token shapes.
3. **How do CUDA Graphs fit dynamic batches?** Graphs prefer fixed shapes and addresses. Engines capture several common batch shapes, pad within bounds, or use stable buffers plus device-side metadata. Unseen shapes may fall back to eager execution.
4. **How should cancellation be handled?** Mark it promptly at a safe iteration boundary, avoid emitting extra tokens, release blocks, and make allocator cleanup exception-safe. Immediate mid-kernel cancellation is generally not practical.
5. **What is head-of-line blocking here?** A large prefill or expensive iteration delays all requests scheduled behind or alongside it. Prompt chunking and separate phase-aware queues bound the blocking interval.

8. Comparison Tables

Feature	Static batching	Dynamic batching	Continuous batching
When batch forms	Before execution	After a short collection window	Before each token/chunk iteration
Membership changes mid-generation	No	No	Yes
Handles variable output lengths	Poorly	Poorly within a formed batch	Well
Implementation complexity	Low	Medium	High
Typical use	Offline jobs	General model APIs	Autoregressive LLM serving
Policy	Strength	Risk	
FCFS	Simple, predictable	Long jobs can block short jobs	
Shortest estimated job	Low mean completion time	Bad estimates and starvation	
Priority	Supports service tiers	Low-priority starvation	
Round-robin/token quota	Fair progress	May reduce maximum throughput	

9. Common Mistakes

- Using “dynamic batching” and “continuous batching” as exact synonyms.
- Measuring request throughput without accounting for different token counts.
- Maximizing batch size while ignoring TTFT, TPOT, and p99 latency.
- Treating prompt tokens and decode tokens as equal-cost units.
- Ignoring KV-cache capacity in scheduler admission.
- Assuming the CPU scheduler is the only overhead; ragged GPU kernels and metadata also matter.
- Forgetting cancellations and stop sequences in the request lifecycle.

10. Edge Cases / Special Cases

- A single very long prompt can dominate an iteration unless prefill is chunked.
- Low arrival rates may never create useful batches; waiting deliberately adds latency.
- Different sampling settings can often share the model forward pass but require separate sampling logic.
- Requests using different adapters may be incompatible or require adapter-aware batching.
- Beam search expands one request into multiple active sequences.
- Multi-GPU workers must make consistent scheduling decisions to avoid collective deadlock.
- Streaming backpressure can require bounded output queues even when GPU generation continues.

11. How to Explain in Interview

“Continuous batching reschedules at token boundaries. After each model iteration, finished sequences leave and waiting sequences enter, so short requests do not leave GPU slots idle behind a long request. Production schedulers use token and KV-memory budgets, and they balance throughput against TTFT and inter-token latency, especially when mixing large prefills with decode.”

12. Quick Revision Notes

- **Definition:** iteration-level changing of batch membership.
- **Main win:** fills slots released by early-finishing requests.
- **Key resource:** scheduled tokens plus KV-cache blocks, not request count alone.
- **Main trade-off:** throughput versus TTFT/TPOT/tail latency.
- **Common comparison:** static/dynamic batching form a batch once; continuous batching reforms it repeatedly.
- **Trap:** one huge prefill can still block interactive decode.

13. Practice Tasks

1. Simulate static and continuous batching for requests with different arrival and output lengths; compare idle slots and completion time.
2. Implement FCFS and shortest-estimated-job schedulers and construct a starvation example.
3. Add a fixed KV-block budget and decide when to admit, preempt, or reject.
4. Plot throughput, TTFT, and TPOT while sweeping maximum scheduled tokens.
5. Design a chunked-prefill policy for chat traffic mixed with document summarization.

14. Final Cheat Sheet

Item	Remember
Core definition	Rebuild the active batch at token/chunk boundaries
Why it matters	Variable-length requests otherwise waste batch slots
Most asked	Static vs continuous, token budget, prefill interference, fairness
Comparisons	FCFS vs priority; throughput vs TTFT/TPOT; static vs dynamic vs continuous
One-line answer	"Continuous batching keeps the GPU busy by admitting and retiring requests between decode iterations."

Paged Attention

1. Overview

Definition. Paged attention is an attention implementation that stores each sequence's KV cache in fixed-size, non-contiguous blocks and uses a block table to find them, much like virtual memory maps virtual pages to physical frames.

Traditional cache allocation often reserves one contiguous region for a sequence's maximum possible length. Real requests have unpredictable lengths, so this wastes memory and creates fragmentation. Paging allocates blocks only as tokens arrive, lets freed blocks be reused, and enables multiple sequences or beams to share prefix blocks.

Paged attention is used in high-throughput LLM serving engines, especially with continuous batching. Interviewers ask about it because it joins GPU kernels, memory management, fragmentation, indirection, and operating-system paging concepts in one practical design.

2. Core Idea

Analogy. A book's logical pages appear consecutive to the reader, although the library may store them on different shelves. A catalog maps page number to shelf location. Likewise, attention sees logical token positions $0 \dots L-1$, while their K/V data may live in unrelated GPU blocks.

```
Logical blocks for request R:  [0] [1] [2] [3]
Block table R:                 [7, 2, 9, 4]
Physical KV pool:              ... block 2 ... block 4 ... block 7 ... block 9
```

If each block holds 16 tokens, logical token 37 belongs to logical block $37 / 16 = 2$ and offset $37 \% 16 = 5$; the table maps logical block 2 to physical block 9.

Step by step:

1. Create a per-request block table.
2. Allocate a physical block from a shared pool when the request needs space.
3. Append new K/V data to the current block.

4. The attention kernel walks logical positions, looks up physical blocks, and loads K/V vectors.
5. On completion, return all owned blocks to the free list.
6. For shared prefixes or beams, reference the same read-only blocks and copy only when a writer would modify a shared block.

Paging does not change the attention equation. It changes the cache's physical storage and the kernel's address calculation.

3. Important Subtopics

Logical versus physical blocks

Logical order belongs to a sequence; physical location belongs to the allocator. The block table provides the mapping. Interview angle: this is application-managed paging, not GPU hardware virtual memory doing everything automatically.

Internal and external fragmentation

Fixed-size blocks largely avoid external fragmentation because any free block can serve any request. Only the final partially filled block wastes space, which is internal fragmentation. With block size B , average tail waste is roughly $B/2$ tokens under a uniform remainder assumption.

Block size

Smaller blocks reduce tail waste and make prefix sharing finer-grained, but increase table size, allocation operations, and address-translation/kernel overhead. Larger blocks do the opposite. There is no universal best value.

Block table and metadata

Each active sequence needs length, block IDs, and sometimes reference counts. Metadata must be updated safely and made visible to the GPU before launch. Efficient implementations stage compact tables in device memory.

Copy-on-write sharing

Beam search or parallel samples can share completed prefix blocks. If two sequences share a partially filled final block and one appends, the engine creates a private copy or allocates a new block to prevent corruption.

Kernel implications

The attention kernel cannot assume $\text{base} + \text{token} * \text{stride}$ across the entire cache. It maps token ranges through a table, loads blocks with coalesced accesses, and combines partial softmax statistics correctly across blocks.

4. Real-World Example

An LLM server has enough free KV memory for 10,000 tokens, but those free regions are split between requests. A contiguous allocator may reject a new 4,000-token reservation because no single region is large enough. A paged allocator can give the request 250 arbitrary 16-token blocks. If the request stops after 900

tokens, it consumed only 57 blocks rather than a maximum-length reservation, and all blocks return directly to the common pool.

5. Diagrams / Mental Models

Request A tokens: 0.....15 | 16.....31 | 32..37

logical blocks: A0 A1 A2

block table: 5 1 8

GPU pool:

block 0 [free]

block 1 [A:16..31]

block 2 [B:0..15]

...

block 5 [A:0..15]

block 8 [A:32..37 + unused tail]

Waste source	Contiguous reservation	Fixed-size paging
Unused maximum length	Potentially large	Not allocated yet
Holes between allocations	External fragmentation	Mostly eliminated
Last allocation unit	Depends on allocator	At most one partial block per sequence
Addressing	Simple base + offset	Block-table lookup + offset

6. Common Interview Questions

#	Question and answer	Interviewer expects	Common mistake
1	What problem does paged attention solve? KV-cache over-reservation and fragmentation under variable sequence lengths.	Memory-utilization motivation.	Saying it reduces model-weight size.
2	Does it approximate attention? No; it computes the same attention over K/V stored through an indirection layer.	Logical result unchanged.	Confusing paging with sparse/local attention.
3	Why “paged”? Logical token blocks map to arbitrary physical KV blocks, analogous to virtual pages and frames.	Mapping analogy and its limits.	Claiming CPU page faults drive each lookup.
4	What is in a block table? Physical block IDs in logical sequence order, plus separate length metadata.	Address translation.	Storing token text instead of K/V locations.
5	What fragmentation remains? Internal waste in the last partially filled block and metadata overhead.	External vs internal fragmentation.	Claiming zero waste.

#	Question and answer	Interviewer expects	Common mistake
6	How does block size affect performance? Small blocks save memory; large blocks reduce metadata/indirection and may improve kernel locality.	Explicit trade-off.	Picking the smallest block unconditionally.
7	How are prefixes shared? Reference-count immutable full blocks; copy on write when a shared block must change.	Safe sharing semantics.	Letting branches append into the same block.
8	How does the kernel read non-contiguous data? It translates each logical block through the table, then performs regular coalesced loads inside the physical block.	Indirection plus local regularity.	Saying accesses become random per scalar.
9	How does this enable continuous batching? Blocks can be allocated/freed incrementally as requests enter, grow, finish, or preempt.	Allocator-scheduler connection.	Treating the two concepts as identical.
10	What overhead does paging add? Tables, reference counts, allocator work, address calculations, and less-simple kernels.	Balanced evaluation.	Saying paging is free because OSes use it.

7. Deep-Dive Questions

- How can softmax be computed block by block without storing all scores?** Maintain an online maximum and normalizing sum. When a new block has maximum m_b , rescale previous accumulators to $\max(m_{old}, m_b)$, add the new exponentials, and similarly rescale/add the weighted-value accumulator. This is mathematically equivalent up to floating-point rounding.
- What concurrency bugs can occur in the allocator?** Double frees, leaked blocks, stale tables, incorrect reference counts, and reuse while an earlier GPU kernel still reads a block. Streams/events or epoch-based lifetimes must order reuse after device work completes.
- How does preemption work with pages?** The scheduler can keep block IDs resident, copy selected blocks to host storage, or free them and later recompute the prefix. Fine-grained blocks make the unit of movement/reclamation manageable.
- Why are partially filled shared blocks tricky?** A branch append would modify memory visible to another sequence. Seal/share only full blocks, or use reference counts and copy-on-write for the tail.
- Could GPU hardware virtual memory replace block tables?** It can help map virtual ranges, but mapping granularity, remapping cost, portability, sharing semantics, and kernel integration often make explicit application-level blocks more controllable.

8. Comparison Tables

Property	Contiguous attention cache	Paged attention cache
Physical layout	One contiguous sequence region	Fixed-size blocks anywhere in pool
Growth	Resize or reserve maximum	Allocate another block

Property	Contiguous attention cache	Paged attention cache
External fragmentation	Can be high	Low
Tail waste	Reservation-dependent	Less than one block per sequence
Prefix/beam sharing	Requires aliasing/copies	Natural at block granularity
Kernel addressing	Simpler	Extra indirection
OS concept	Paged-attention analogue	Important difference
Virtual page	Logical KV block	Usually chosen by serving engine
Physical frame	GPU KV pool block	Contains structured K/V tensors
Page table	Per-sequence block table	Read explicitly by attention kernel
Copy-on-write	Shared prefix blocks	Managed by application/runtime
Page fault	Allocate/swap/recompute event	Generally scheduled explicitly, not transparent hardware faulting

9. Common Mistakes

- Calling paged attention a new attention algorithm rather than a memory-layout/execution technique.
- Claiming it removes all fragmentation.
- Ignoring reference counts when sharing blocks.
- Choosing block size from memory waste alone and ignoring kernel performance.
- Assuming logical adjacency guarantees physical adjacency.
- Forgetting that length metadata prevents reading unused tail entries.
- Reusing freed blocks before all asynchronous GPU readers finish.

10. Edge Cases / Special Cases

- A sequence exactly divisible by block size has no partial tail block.
- Very short requests may pay more metadata overhead relative to useful cache.
- Prefix matches ending mid-block complicate sharing; engines may share only complete blocks.
- Windowed attention can recycle old logical blocks, but positions must remain logically correct.
- Tensor-parallel ranks need corresponding local blocks and consistent request/block-table ordering.
- Allocator exhaustion requires admission control or preemption even if model weights still fit.
- Block tables themselves need capacity bounds for maximum context length.

11. How to Explain in Interview

"Paged attention keeps a sequence's KV cache in fixed-size GPU blocks and maps logical token blocks through a per-sequence table. It preserves exact attention while avoiding maximum-length reservations and most external fragmentation. The trade-off is block metadata and address indirection, while the benefits include higher concurrency, incremental allocation, and copy-on-write prefix sharing."

12. Quick Revision Notes

- **Definition:** block-table-addressed, non-contiguous KV cache.
- **Main win:** allocate on demand and reuse arbitrary free blocks.
- **Waste:** usually only the partially filled tail block per sequence.
- **Trade-off:** block size balances fragmentation against metadata/kernel overhead.
- **Sharing:** reference-count full prefix blocks; copy on write.
- **Trap:** it is exact attention, not sparse attention.

13. Practice Tasks

1. Implement a block pool with **allocate**, **free**, **fork**, and reference counts.
2. Translate logical token indices to block/offset pairs for several block sizes.
3. Simulate random sequence arrivals and compare reserved, contiguous, and paged memory utilization.
4. Implement online softmax over score chunks and compare with a full softmax.
5. Draw the block tables before and after a beam forks and appends a token.

14. Final Cheat Sheet

Item	Remember
Core definition	Exact attention over KV cache stored in mapped fixed-size blocks
Why it matters	Better memory utilization means more concurrent sequences
Most asked	Fragmentation, block size, block table, copy-on-write, kernel overhead
Comparisons	Contiguous vs paged; external vs internal fragmentation
One-line answer	"Paged attention virtualizes the KV cache so variable-length requests allocate reusable GPU blocks on demand."

Dynamic Batching

1. Overview

Definition. Dynamic batching collects independently arriving requests for a short time or until size limits are met, combines compatible inputs into one batch, executes them together, and returns individual results.

Unlike a fixed offline batch, the server does not know all inputs in advance. It makes a runtime decision based on queue state, maximum wait time, batch size, token count, shapes, and request compatibility. Dynamic batching is widely used for image classifiers, encoders, embedding models, rerankers, and LLM prefill. For autoregressive generation it is useful, but continuous batching goes further by changing membership during generation.

Interviewers ask about it because it exposes the basic throughput-latency trade-off in online GPU serving: waiting can create more efficient work, but waiting itself adds latency.

2. Core Idea

Analogy. An elevator can depart for every person immediately, minimizing that person's wait but wasting capacity, or wait briefly to collect passengers going in the same direction. A good policy waits only within a latency budget.

```
request arrivals: A---B-C-----D
collection window: [A B C]          [D]
GPU executions:      batch ABC      D
```

Step by step:

1. A request is validated and placed in a compatibility queue.
2. The batcher starts or observes a maximum-delay timer.
3. It adds requests while batch-size, token, and memory limits permit.
4. It pads, packs, or creates ragged metadata as the model requires.
5. One batched GPU call runs.
6. Outputs are split and delivered to the original request futures.

The policy generally flushes when either a size/work threshold is reached or the oldest request hits its allowed delay.

3. Important Subtopics

Batching window

A longer window usually forms larger batches but adds queueing latency. Traffic rate matters: at high load, batches fill quickly without much deliberate delay; at low load, waiting may not find another request.

Compatibility grouping

Requests may differ by model, adapter, dtype, input shape, decoding mode, or SLO. Only requests executable by the same kernel/model configuration can share a batch. Too many queues fragment traffic and reduce batching opportunities.

Padding versus packing

Padding extends every input to the longest length and can waste compute. Packing concatenates valid tokens and supplies offsets/lengths, requiring model and kernels that understand ragged inputs. Bucketing similar lengths is a simpler compromise.

Batch constraints

Maximum request count is insufficient for variable-size inputs. Production batchers also cap total tokens, bytes, pixels, or estimated compute. This prevents one large item from causing out-of-memory or excessive latency.

Queueing and backpressure

When arrival rate exceeds service rate, the queue grows without bound unless the server rejects, sheds, or routes load. Batching improves capacity but does not repeal queueing theory.

Response demultiplexing

Outputs must map back to original requests, preserve errors/cancellations, and avoid one failed item corrupting the whole batch. Interview angle: batch execution is shared, request lifecycle is not.

4. Real-World Example

An embedding endpoint receives sentences individually. Running a GPU kernel per sentence spends much of its time on launch overhead and underfilled matrix multiplications. The server waits up to 3 ms or 64 requests, whichever comes first, then packs at most 8,192 tokens. During peak traffic the batch fills nearly instantly; during quiet periods the timer bounds user-visible delay. Sentences are bucketed by length to reduce padding.

5. Diagrams / Mental Models

```

compatible queue
A -----> [A]
B -----> [A B] -- size/token/delay trigger --> collate --> GPU --> split
C -----> [A B C]
                        / | \
                        A  B  C

```

Increase this	Usually improves	Usually worsens
Max batch size	GPU efficiency, throughput	Memory use, batch service time
Max queue delay	Average batch fullness	Per-request latency
Length bucketing	Padding efficiency	Queue fragmentation/wait
Token cap	Predictability and OOM safety	Peak batch utilization if too low

6. Common Interview Questions

#	Question and answer	Interviewer expects	Common mistake
1	What is dynamic batching? Runtime grouping of compatible requests that arrive independently.	Online queue and flush policy.	Describing a fixed training batch.
2	Why batch on a GPU? Larger matrix operations use parallel hardware better and amortize launch/framework overhead.	Utilization and amortization.	Saying batching reduces total mathematical work.
3	What is the latency trade-off? Requests wait to form a batch; larger batches can also take longer to execute.	Queue delay plus service time.	Considering only queue delay.
4	When should a batch flush? At work/size limit or when the oldest request reaches its wait deadline.	Dual trigger.	Waiting forever for a full batch.

#	Question and answer	Interviewer expects	Common mistake
5	Why use a token limit? Input lengths vary, so request count poorly predicts memory and compute.	Work-aware budgeting.	Letting one huge prompt into an otherwise full batch.
6	Padding or packing? Padding is simple but wasteful; packing needs ragged support but executes only valid tokens.	Practical comparison.	Claiming packing has no metadata cost.
7	How does it differ from continuous batching? Dynamic batching forms a batch before a request-level execution; continuous batching changes membership between generation iterations.	Precise lifecycle distinction.	Treating the names as interchangeable.
8	How do you choose the delay? Start from the latency SLO and measured arrival/service distributions; tune p95/p99.	Workload-driven choice.	Selecting an arbitrary constant from another service.
9	What is head-of-line blocking? An incompatible or huge oldest request can delay smaller compatible work depending on queue policy.	Queue organization and bypass rules.	Assuming FCFS is always fair and efficient.
10	What happens under overload? Apply bounded queues, admission control, load shedding, autoscaling, or routing.	Backpressure.	Increasing the batching window indefinitely.

7. Deep-Dive Questions

1. **How would you model latency?** Approximately **queue wait + collation/transfer + batch execution + demultiplexing**. Arrival rate affects batch fullness and queueing nonlinearly; measure distributions rather than only a mean.
2. **Can batching reduce per-request latency?** Yes under an already-saturated system: higher throughput can shorten the queue enough to offset a small collection delay. At low load, it normally adds latency.
3. **How do shape buckets help?** They group similar lengths/shapes, reducing padding and supporting optimized fixed-shape kernels or CUDA Graphs. Too many buckets reduce traffic per queue and increase waits.
4. **How should deadlines be handled?** Track each request's remaining budget, flush before the oldest deadline, avoid adding work that would predictably violate it, and prioritize or reject requests when capacity is insufficient.
5. **What if one item causes a batch-level GPU error?** Validate per request before collation where possible; fail all affected requests consistently for a true batch failure, release buffers, and isolate malformed items at the trust boundary.

8. Comparison Tables

Property	Fixed/offline batch	Dynamic batch	Continuous batch
Inputs known beforehand	Yes	No	No
Runtime collection window	No	Yes	Often admission queue rather than one fixed window
Membership after execution begins	Fixed	Fixed	Changes at iteration boundaries
Best fit	Training/offline inference	Encoders, classifiers, prefill	Autoregressive decode
Main concern	Device utilization	Wait vs batch efficiency	Scheduling, KV memory, token latency
Input handling	Advantage	Disadvantage	
Pad to maximum	Simple dense tensor/kernels	Wasted work and memory	
Pack/ragged	Only valid elements processed	Offsets and specialized kernels	
Length buckets	Simple compromise	More queues and possible wait	

9. Common Mistakes

- Calling any non-fixed batch “continuous.”
- Tuning average latency while missing p99 deadline violations.
- Capping request count but not total work.
- Padding wildly different input lengths into one batch.
- Forgetting compatibility dimensions such as model adapter or dtype.
- Leaving queues unbounded during overload.
- Assuming the largest possible batch is the most efficient batch.

10. Edge Cases / Special Cases

- At batch size one, the batcher should flush promptly rather than always wait the full window if the SLO demands it.
- One oversized request may require a dedicated path or rejection.
- Cancellation while queued should remove the request without disturbing others.
- Cancellation after launch may discard that item's output, but generally cannot cheaply shrink the running batch.
- Variable output shapes complicate demultiplexing.
- Multi-tenant queues need quotas so high-volume tenants do not monopolize batches.
- Cold starts and model swaps can dominate all batching gains.

11. How to Explain in Interview

“Dynamic batching waits briefly to group compatible online requests into a larger GPU operation. It improves throughput by amortizing launches and creating efficient matrix sizes, but adds queueing and padding costs.

A practical batcher flushes on either a token/size threshold or the oldest request's deadline, uses bounded queues, and is distinct from continuous batching, which changes membership during autoregressive generation.”

12. Quick Revision Notes

- **Definition:** runtime collection of independently arriving requests.
- **Flush:** work limit reached or oldest request deadline reached.
- **Budget:** use tokens/bytes/compute, not only number of requests.
- **Input strategy:** padding is simple; packing is efficient; bucketing is a compromise.
- **Comparison:** dynamic forms once; continuous reforms during generation.
- **Trap:** batching improves throughput but may increase latency.

13. Practice Tasks

1. Implement an asynchronous batcher with maximum delay, count, and token limits.
2. Generate random input lengths and compare padded versus packed token work.
3. Simulate low, medium, and overload arrival rates; plot batch size and p99 latency.
4. Add length buckets, then identify when queue fragmentation outweighs padding savings.
5. Design cancellation and error propagation for requests sharing one GPU call.

14. Final Cheat Sheet

Item	Remember
Core definition	Collect compatible online requests into one execution batch
Why it matters	Better GPU utilization and amortized overhead
Most asked	Flush policy, latency trade-off, padding, overload, continuous-batching difference
Comparisons	Fixed vs dynamic vs continuous; padding vs packing
One-line answer	“Dynamic batching trades a bounded amount of waiting for a more efficient shared GPU execution.”

Prefix Caching

1. Overview

Definition. Prefix caching stores and reuses the KV cache produced by an identical token prefix across requests, avoiding repeated prefill computation for that prefix.

Many LLM calls begin with the same system prompt, tool definitions, few-shot examples, document template, or conversation history. Because a deterministic Transformer produces the same K/V tensors for the same model configuration and token prefix, those tensors can be shared. Prefix caching is used in chat platforms, agent systems with large tool schemas, retrieval systems with repeated documents, and APIs that expose prompt-caching discounts.

It matters because prefill of a long repeated prefix consumes GPU compute, memory bandwidth, and time to first token. Interviewers ask about it to test correctness of cache keys, trie/hash design, eviction, multi-tenancy, positional semantics, and the difference between reusing computation and reusing generated answers.

2. Core Idea

Analogy. A compiler need not parse an unchanged common header separately for every source file if it can reuse a valid precompiled form. The reuse is safe only when the header and relevant compiler settings match exactly.

Example:

```
Request 1 tokens: [SYSTEM][TOOLS.....][user: weather?]  
Request 2 tokens: [SYSTEM][TOOLS.....][user: calendar?]  
                  ^  
            identical prefix ends here
```

Step by step:

1. Tokenize the request using the exact production tokenizer and template.
2. Divide the token prefix into cacheable blocks or incrementally hash tokens.
3. Include model/configuration identity in the cache namespace.
4. Look up the longest exact prefix already present.
5. Attach/reference its KV blocks.
6. Prefill only the unmatched suffix.
7. Add newly completed blocks to the prefix index according to policy.
8. Evict entries when memory pressure, TTL, or namespace invalidation requires it.

This does **not** reuse the model's final answer. Sampling continues normally from the reused state, so different suffixes or random choices can produce different outputs.

3. Important Subtopics

Exact token matching

The reusable unit is a sequence of token IDs, not visually similar text. Whitespace, chat templates, Unicode normalization, tokenizer version, and tool ordering can change tokenization. Interview angle: hash tokens after final prompt construction.

Cache key and namespace

A safe key includes model weights/version, adapter identity, token IDs, and any setting that changes hidden-state/KV computation. Sampling temperature need not invalidate prefix KV because it acts after logits, but attention/position/model changes do.

Longest-prefix lookup

A trie naturally represents token prefixes. Block hashing is often more compact: hash each block together with the parent hash, then find the longest chain of hits. Block-granular indexing aligns well with paged KV storage.

Cache ownership and isolation

Private prompts must not become observable across tenants through direct access or timing/cost side channels. Systems may scope caches by tenant, trust domain, or explicit public prefixes. Security can outweigh hit rate.

Eviction

GPU memory is scarce and prefix blocks compete with active request caches. LRU, LFU, TTL, size-aware value, or recomputation-cost-aware eviction may be used. Reference-counted blocks actively used by requests cannot be evicted.

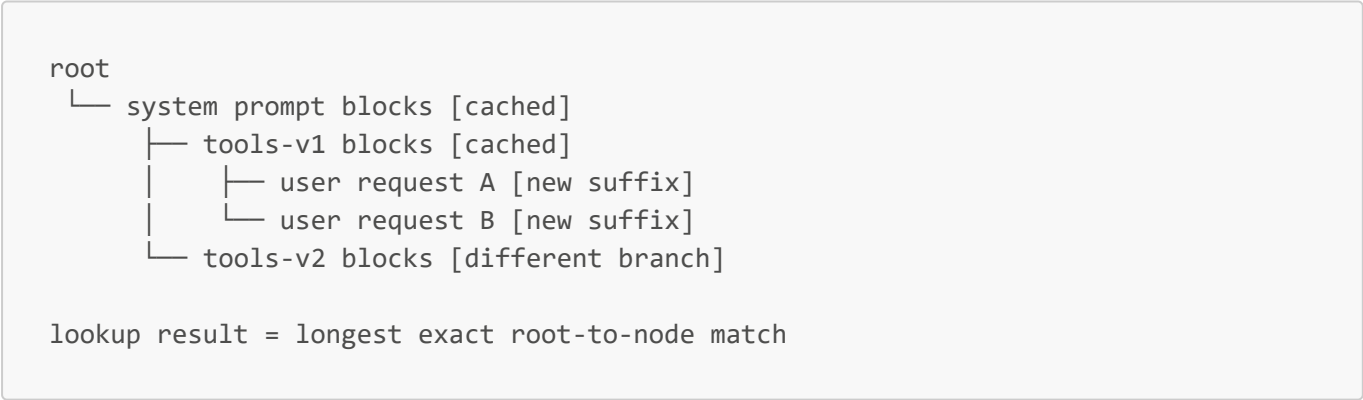
Partial-block matches

If block size is 16 tokens and only 10 tokens match, an engine may reuse only complete matching blocks and recompute the tail. Finer matches save more compute but complicate storage and kernels.

4. Real-World Example

An agent platform includes a 6,000-token system prompt and tool catalog on every request. Without prefix caching, 100 requests prefill those 6,000 tokens 100 times. With a tenant-scoped cached prefix, the first request pays the prefill cost; later requests reference those KV blocks and process only their unique user messages. Time to first token drops, while the scheduler preserves the cached blocks only as long as their saved recomputation value justifies GPU memory.

5. Diagrams / Mental Models



Event	Cache action
Exact full-block prefix hit	Reference cached blocks; skip their prefill
Match ends inside block	Reuse completed matching blocks; recompute remainder in simple designs
Model/adaptor changes	Use another namespace or invalidate
Active request uses entry	Pin/reference-count; do not evict
Memory pressure	Evict unreferenced low-value prefixes

6. Common Interview Questions

#	Question and answer	Interviewer expects	Common mistake
1	What is prefix caching? Reuse of K/V states for an identical starting token sequence.	Cached computation, not cached response.	Saying it returns the previous answer.
2	What must match? Token IDs and all model/configuration state affecting K/V.	Exactness and namespace.	Comparing raw strings only.
3	What performance metric improves most? Prefill work and often TTFT for repeated long prefixes.	Work saved is proportional to matched prefix.	Claiming decode TPOT always improves.
4	How do you find a match? Trie or chained block hashes support longest-prefix lookup.	Incremental prefix structure.	Hashing only the entire prompt, which misses partial reuse.
5	Why pair it with paged attention? Cached prefixes can share immutable KV blocks by reference.	Block-granular reuse and copy-on-write.	Copying the cached prefix into every request.
6	What invalidates the cache? Model/adaptor/tokenization/template or relevant execution changes.	Comprehensive identity.	Including only model name while weights changed.
7	Does temperature affect validity? No for prefix K/V; temperature changes sampling after logits, not prefix hidden states.	Stage separation.	Invalidating for every sampling parameter.
8	How do you evict? Evict unreferenced entries using recency/frequency/size/recompute value under memory pressure.	Active pinning and value-aware policy.	Freeing blocks still used by a kernel.
9	What are security risks? Cross-tenant information/timing leakage and unintended private-prompt retention.	Isolation and policy.	Maximizing global hit rate without trust boundaries.
10	When is it not useful? Mostly unique or short prompts, frequently changing prefixes, or memory pressure making retention cost exceed saved prefill.	Workload-dependent value.	Assuming every chat has high reuse.

7. Deep-Dive Questions

- How can a chained block hash prevent false reuse?** Compute $H_i = \text{hash}(H_{i-1}, \text{token_block}_i, \text{namespace})$. A hit is meaningful only with the same parent chain. Use a strong hash and verify tokens on collision-sensitive paths rather than relying on a weak digest.
- How do positional encodings affect reuse?** A prefix reused at the same logical positions is normally valid. Reusing a substring at a different position is not ordinary prefix caching because RoPE/absolute positions change K/V values; it needs model-specific transformations or recomputation.

3. **How would you estimate cache value?** Approximate saved prefill cost times expected future hit probability, divided by occupied bytes, while considering eviction/reload costs. Long, frequently reused prefixes have high value.
4. **Can prefixes be shared across tensor-parallel replicas?** Every rank needs the corresponding shard. Sharing within one replica is simple; cross-replica reuse needs routing to the owning replica or transferring/distributing sharded cache, which may cost more than recomputation.
5. **What happens when a LoRA adapter changes?** Since adapter-modified projections/hidden states can change K/V, include adapter identity/version in the namespace. Base-model prefixes are not automatically reusable across arbitrary adapters.

8. Comparison Tables

Feature	KV cache	Prefix cache	Response cache
Reused object	K/V for one active sequence	K/V across matching requests	Final generated result
Match requirement	Same sequence history	Exact prefix and model state	Usually exact semantic/request key
Sampling still runs	Yes	Yes	No on hit
Main benefit	Faster incremental decode	Faster repeated prefill	Avoid entire inference
Main risk	Capacity	Staleness/isolation/memory	Stale or semantically wrong answer
Index	Strength	Limitation	
Token trie	Natural longest-prefix match	Node/metadata overhead	
Full-prompt hash	Simple	Only complete matches	
Chained block hashes	Matches paged storage; partial prefix hits	Block granularity and collision handling	

9. Common Mistakes

- Calling prefix caching “semantic caching”; ordinary prefix reuse requires exact tokens.
- Forgetting model and adapter versions in the namespace.
- Assuming raw string equality guarantees token/template equality.
- Claiming it improves every decode step rather than primarily eliminating repeated prefill.
- Copying shared KV data and losing the memory benefit.
- Caching private prompts globally without isolation analysis.
- Evicting blocks that active requests still reference.

10. Edge Cases / Special Cases

- A shared prefix ending in a partially filled block may need tail copying or recomputation.
- Chat templates may insert timestamps or changing metadata that destroy matches.

- Tool definitions in nondeterministic order reduce hit rate despite equivalent meaning.
- Prefixes near the context limit may leave insufficient room for requested output.
- Cache hits can reveal that another tenant used a prefix through timing unless namespaces isolate them.
- Multimodal prefixes include encoded image/audio states whose identity and preprocessing version matter.
- Cached blocks must be invalidated after hot model-weight updates.

11. How to Explain in Interview

“Prefix caching reuses the KV blocks generated for an exact token prefix across requests. The server finds the longest valid match, references those immutable blocks, and prefills only the suffix. A correct key includes model, adapter, tokenizer/template, and tokens; practical designs use block hashes or tries, reference counting, eviction, and tenant isolation.”

12. Quick Revision Notes

- **Definition:** cross-request reuse of prefix KV states.
- **Match:** exact token sequence at the same positions, under compatible model state.
- **Primary benefit:** lower prefill cost and TTFT.
- **Common structure:** trie or chained hashes over KV blocks.
- **Main trade-off:** valuable GPU memory versus future recomputation saved.
- **Trap:** it does not cache or force the generated answer.

13. Practice Tasks

1. Implement longest-prefix lookup with a token trie.
2. Implement chained hashes for 16-token blocks and test diverging prompts.
3. Build an LRU cache with pin/reference counts and prove active blocks cannot be evicted.
4. Calculate saved prefill tokens and hit rate for repeated tool-schema prompts.
5. Write cache-key test cases covering adapter versions, templates, tokenizers, and temperature.

14. Final Cheat Sheet

Item	Remember
Core definition	Reuse identical-prefix KV computation across requests
Why it matters	Long common prompts otherwise repeat expensive prefill
Most asked	Exact-match rule, cache key, trie/hash, eviction, isolation
Comparisons	Prefix cache vs per-request KV cache vs response cache
One-line answer	“Prefix caching skips repeated prefill by sharing valid immutable KV blocks for the longest exact token prefix.”

Quantization

1. Overview

Definition. Quantization represents model tensors and sometimes computations with fewer bits than FP32/FP16, using a mapping between real values and a smaller discrete numeric set.

LLM inference uses quantization for weights, activations, KV caches, and occasionally communication. Common formats include INT8, INT4, FP8, and hardware-specific low-bit floating-point types. It reduces model footprint and memory traffic, often increasing throughput and allowing larger models or batches on the same GPUs. The cost is quantization/dequantization overhead and possible accuracy loss.

Interviewers ask about it because a strong answer must connect numerical representation, scales/zero-points, calibration, GPU kernel support, memory bandwidth, accuracy, and the distinction between weight-only and weight-activation quantization.

2. Core Idea

For affine integer quantization:

```
q = clamp(round(x / scale) + zero_point, q_min, q_max)
x_approx = scale * (q - zero_point)
```

For symmetric signed quantization, `zero_point = 0` and a common scale is:

```
scale = max(abs(x)) / (2^(bits-1) - 1)
```

Analogy. A detailed elevation map may store centimeter precision, but a hiking app can store heights in half-meter steps. The file is smaller; nearby heights collapse to the same code, and extremes need a chosen range.

Small example: quantize `[-1.0, -0.2, 0.3, 0.9]` to signed 4-bit values `[-7, 7]`. With scale `1/7`, codes are approximately `[-7, -1, 2, 6]`; dequantized values are `[-1.0, -0.143, 0.286, 0.857]`. The differences are quantization error.

Step by step:

1. Choose which tensors and operations to quantize.
2. Choose format, granularity, and range-estimation method.
3. Compute scale(s), and zero-point(s) for asymmetric integer schemes.
4. Round and clamp values into the representable range.
5. Store packed low-bit values and metadata.
6. At inference, kernels dequantize while loading or use native low-precision matrix instructions.
7. Validate task quality and benchmark end-to-end latency/throughput.

3. Important Subtopics

Weight-only quantization

Weights are stored at low precision while activations remain FP16/BF16. It reduces weight bandwidth and capacity and is comparatively easy to deploy. A fused GEMM dequantizes weights inside the kernel. It helps memory-bound inference, especially decode.

Weight-and-activation quantization

Both operands use low precision, enabling INT8/FP8 tensor-core paths and potentially larger speedups. Activation outliers and dynamic ranges make accuracy and calibration harder.

Post-training quantization versus quantization-aware training

Post-training quantization (PTQ) transforms an existing model, perhaps with calibration data. Quantization-aware training (QAT) simulates quantization during fine-tuning/training so weights adapt, usually preserving quality better at greater cost.

Granularity

Per-tensor uses one scale, per-channel uses one scale per output/input channel, and group-wise uses one scale per small group (for example 32 or 128 weights). Finer granularity represents local ranges better but adds metadata and kernel work.

Symmetric versus asymmetric

Symmetric quantization centers at zero and simplifies arithmetic. Asymmetric quantization adds a zero-point and can use range more effectively for skewed nonnegative values, but introduces correction terms.

Outliers and clipping

A few extreme values force a large scale and waste most quantization levels. Clipping, channel-wise scales, keeping outlier channels in higher precision, or activation smoothing can improve accuracy.

KV-cache quantization

Quantizing K/V reduces a memory cost that grows with tokens and concurrency. It has separate scales, kernels, and error behavior from weight quantization and can improve decode bandwidth/capacity.

4. Real-World Example

A 70-billion-parameter model requires roughly 140 GB for FP16 weights alone. Four-bit weight storage is about 35 GB before scales and packing overhead, making deployment on fewer GPUs possible. During decode, each generated step repeatedly streams model weights; reducing those bytes can improve throughput if fused low-bit kernels efficiently feed tensor cores. The team still evaluates perplexity and downstream tasks because fitting the model does not guarantee acceptable quality.

5. Diagrams / Mental Models

```

FP value ----divide by scale----> round/clamp ----> low-bit code
      ^                                     |
      |----- multiply by scale <-----|
              approximate value

```

Error sources:

rounding: value lies between representable levels
clipping: value lies outside chosen range

Scheme	Stored weights	Activations	Typical benefit	Main challenge
FP16/BF16	16-bit	16-bit	Strong baseline	Capacity/bandwidth
W8A16	INT8	FP16/BF16	Lower weight traffic	Dequantization/kernel quality
W4A16	INT4	FP16/BF16	Much smaller weights	Accuracy and packing overhead
W8A8	INT8	INT8	Native low-precision GEMM	Activation calibration/outliers
FP8	FP8	FP8/FP16 variants	Tensor-core acceleration with exponent range	Scaling and hardware support

6. Common Interview Questions

#	Question and answer	Interviewer expects	Common mistake
1	What is quantization? Mapping tensors to fewer representable values/bits with scale metadata.	Storage and approximate reconstruction.	Saying it is ordinary compression with exact recovery.
2	Why can it speed inference? Fewer bytes moved and sometimes faster low-precision tensor-core arithmetic.	Roofline/hardware perspective.	Assuming 4-bit always gives exactly 4× speedup over FP16.
3	What are scale and zero-point? Scale sets real-value step size; zero-point maps real zero into integer space.	Formula and symmetric case.	Treating zero-point as a learned bias.
4	Weight-only vs W8A8? Weight-only leaves activations high precision; W8A8 also quantizes activations and can use integer GEMM but is harder.	Deployment and accuracy trade-off.	Inferring activation dtype from weight dtype.
5	Per-tensor vs per-channel/group? Finer scales fit local ranges better but add metadata/complexity.	Accuracy-efficiency trade-off.	Saying per-channel reduces bit width further.
6	PTQ vs QAT? PTQ converts a trained model; QAT trains with simulated quantization error.	Cost versus quality.	Calling calibration full retraining.

#	Question and answer	Interviewer expects	Common mistake
7	Why do outliers hurt? They enlarge the represented range, making steps coarse for most values or causing clipping.	Dynamic-range reasoning.	Saying low-bit integers cannot represent negative numbers.
8	INT8 vs FP8? INT8 has uniform spacing under a scale; FP8 has sign/exponent/mantissa and wider relative dynamic range.	Representation and hardware.	Assuming equal bit count means identical behavior.
9	What should be evaluated? Task quality/perplexity plus end-to-end latency, throughput, memory, and power on target hardware.	Accuracy and systems measurement.	Reporting file size alone.
10	Why might quantization be slower? Unfused dequantization, unsupported shapes/hardware, packing overhead, small batches, or higher-precision fallbacks.	Kernel reality.	Assuming fewer bits automatically means faster execution.

7. Deep-Dive Questions

1. **How does group size affect W4 quantization?** Smaller groups get more precise local scales and usually better accuracy, but require more scale metadata and can reduce kernel efficiency. Larger groups compress metadata and simplify access but expose more range variation.
2. **Why are some layers kept in higher precision?** Embeddings, output heads, normalization, or outlier-sensitive projections can dominate quality loss while contributing a modest fraction of memory. Mixed precision targets low-bit storage where it pays.
3. **How does activation-aware weight quantization work conceptually?** Calibration identifies input channels important to output error; scaling or optimization preserves those channels while quantizing weights. The objective is output reconstruction, not merely minimizing weight error.
4. **What is dynamic activation quantization?** Scales are computed from current runtime activations rather than fixed calibration ranges. It adapts to inputs but adds reduction and scaling overhead.
5. **How would you reason with a roofline model?** Quantization increases effective arithmetic intensity by reducing bytes per operation. A memory-bound kernel may speed up substantially; a compute-bound kernel benefits only if hardware performs the low-precision operations faster and conversion does not dominate.

8. Comparison Tables

Property	INT4	INT8	FP8	FP16/BF16
Bits/value	4	8	8	16
Dynamic range	Scale-dependent, uniform levels	Scale-dependent, uniform levels	Exponent provides wide range	Wide

Property	INT4	INT8	FP8	FP16/BF16
Precision	Lowest	Moderate	Varies with magnitude	Highest here
Typical LLM use	Weight-only	Weights/activations/KV	Activations/weights on newer GPUs	Baseline execution
Metadata/conversion	Often group scales and packing	Scales, optional zero-points	Scaling strategy	Minimal
Granularity	Accuracy	Scale overhead	Kernel simplicity	
Per-tensor	Lowest generally	Lowest	Highest	
Per-channel	High	Medium	Medium	
Per-group	Tunable compromise	Medium/high	Requires optimized grouped loads	

9. Common Mistakes

- Equating model-size reduction with equal latency reduction.
- Evaluating only perplexity or only speed, instead of both.
- Forgetting scale and packing metadata in memory estimates.
- Confusing FP8 with an 8-bit integer.
- Assuming all operations run at the advertised low precision.
- Applying one global scale to outlier-heavy tensors and blaming bit width alone.
- Treating weight, activation, and KV-cache quantization as one setting.

10. Edge Cases / Special Cases

- Zero-range groups need a safe nonzero/default scale to avoid division by zero.
- NaN and infinity handling depends on format and conversion rules.
- Dimensions not divisible by packing/group sizes may need padding or a slower tail path.
- Small matrices may not amortize unpacking/dequantization.
- Saturation from a bad calibration set may appear only on rare production inputs.
- Different FP8 formats trade exponent range against mantissa precision.
- Quantized logits or sampling can alter close token rankings, so output heads are often treated carefully.

11. How to Explain in Interview

“Quantization maps weights or activations to a lower-bit format using scales, and sometimes zero-points, then uses fused kernels to reconstruct or compute with them. It saves capacity and bandwidth and can use faster tensor-core paths, but speedup depends on hardware and fusion, while accuracy depends on granularity, calibration, outliers, and which tensors remain high precision.”

12. Quick Revision Notes

- **Formula:** $q = \text{round}(x/\text{scale}) + \text{zero_point}$; dequantize with $\text{scale} \times (q - \text{zero_point})$.
- **W4A16:** 4-bit weights, 16-bit activations.

- **PTQ/QAT:** convert after training versus train while simulating low precision.
- **Granularity:** smaller groups usually improve accuracy but cost metadata/kernel complexity.
- **Main performance reason:** fewer bytes plus possible faster arithmetic.
- **Trap:** theoretical bit reduction is not guaranteed end-to-end speedup.

13. Practice Tasks

1. Quantize a vector symmetrically to INT8 and INT4; compute maximum and mean error.
2. Compare per-tensor and per-channel quantization on a matrix with one outlier channel.
3. Calculate weight memory for FP16, INT8, and group-wise INT4 including scale metadata.
4. Write a small packed-INT4 dequantization routine and verify odd/tail dimensions.
5. Use a roofline estimate to predict when halving bytes should improve latency.

14. Final Cheat Sheet

Item	Remember
Core definition	Approximate tensors with fewer-bit values plus scaling metadata
Why it matters	Smaller models, less bandwidth, and possibly faster math
Most asked	Scale/zero-point, PTQ vs QAT, W4A16 vs W8A8, outliers, real speedup
Comparisons	INT vs FP8; per-tensor vs per-channel/group; weight vs KV quantization
One-line answer	"Quantization trades controlled numerical error for lower memory traffic, capacity, and sometimes faster GPU arithmetic."

Speculative Decoding

1. Overview

Definition. Speculative decoding uses a cheaper proposer—usually a smaller draft model—to predict several future tokens, then verifies those tokens in parallel with the target model and accepts as many as correctness allows.

Ordinary autoregressive decoding invokes the large target model once per output token. That serial dependency limits latency even when each step underuses the GPU. Speculation spends cheap work to propose k tokens and turns one target-model call into progress of potentially several tokens. Exact acceptance algorithms preserve the target model's output distribution; greedy variants can preserve greedy output.

It is used in interactive LLM serving, on-device/cloud model pairs, self-speculative layer skipping, and multi-token prediction systems. Interviewers ask about it to test probabilistic correctness, acceptance rate, cost models, batching interactions, and the difference between "predicting faster" and "changing the model."

2. Core Idea

Analogy. A junior editor drafts the next paragraph quickly. A senior editor reviews the whole proposal in one pass, accepts the correct beginning, fixes the first disagreement, and discards everything after it because it depended on the wrong word.

Greedy example:

```
draft proposes: [the, blue, car, stopped]
target verifies: [the, blue, bus, ...]
accepted:      [the, blue]
correction:    [bus]
discard:              [stopped]
```

The target processes the proposed token sequence in one forward pass using causal masking. Although positions still depend logically on earlier tokens, GPU matrix operations for verification are parallelized like a short prefill.

For exact sampling, let draft distribution be $q(x)$ and target distribution be $p(x)$ at a position:

```
accept proposed x with probability min(1, p(x) / q(x))
```

On rejection, sample a correction from a normalized residual distribution proportional to $\max(0, p(x) - q(x))$. This rejection-sampling construction makes the final token distribution exactly p , assuming correct implementation.

Step by step:

1. Draft model generates k candidate tokens autoregressively.
2. Target model scores all k candidates in one verification pass.
3. Compare proposal and target probabilities left to right.
4. Accept a consecutive prefix according to the chosen greedy/exact rule.
5. At the first rejection, generate a corrected token and discard dependent later candidates.
6. Commit accepted/corrected KV states and repeat.

3. Important Subtopics

Draft-target agreement

High agreement/acceptance is essential. A very small but inaccurate draft is cheap yet wastes target verification; a large accurate draft may consume most of the saved time. The best draft minimizes total latency, not draft latency alone.

Exact versus approximate methods

Exact speculative sampling uses acceptance correction so output distribution equals the target. Greedy verification accepts while draft tokens equal target argmax. Heuristic acceptance can be faster but may change quality/distribution and must be described honestly.

Verification parallelism

The target evaluates multiple candidate positions together, turning several memory-bound decode steps into a denser operation. It does not skip target validation; it reorganizes it.

KV-cache handling

Draft and target maintain separate caches. Target verification tentatively creates K/V for candidates; only accepted tokens and the correction are committed. Rejected suffix states must be rolled back, masked, or simply not added to the committed length.

Speculation length

Larger k offers more potential tokens per verification but more wasted draft/target work after early rejection. Adaptive policies use recent acceptance rate, entropy, or latency measurements.

Alternative proposers

The proposer can be a smaller model, early-exit layers of the target, extra prediction heads, a prompt n-gram cache, or a tree of candidates. Each changes memory, training, and verification complexity.

4. Real-World Example

A 70B target model is bandwidth-bound during single-token decode. A colocated 7B draft proposes five tokens. The target verifies all five with a single short-sequence forward pass; if it accepts an average of four plus one correction token, each target invocation advances roughly five positions instead of one. The benefit is largest for predictable text such as code or boilerplate and smaller for high-entropy creative output.

5. Diagrams / Mental Models

```

prefix P
|
+-- draft: d1 -> d2 -> d3 -> d4
|
+-- target verification in one pass
    accept d1
    accept d2
    reject d3 -> sample target correction c3
    discard d4

new committed prefix = P + d1 + d2 + c3
  
```

Approximate latency condition:

```

speculation wins when
(draft cost for k + target verification cost + overhead)
----- < target decode cost per
  
```

token
expected committed tokens per round

6. Common Interview Questions

#	Question and answer	Interviewer expects	Common mistake
1	What is speculative decoding? Cheap multi-token proposal followed by target-model parallel verification.	Propose, verify, accept prefix, correct.	Saying the small model's output is returned unchecked.
2	Why can it be faster? It amortizes a target invocation over multiple accepted tokens and makes verification denser.	Serial decode bottleneck.	Saying it reduces target model parameter count.
3	Does it change output quality? Exact algorithms preserve the target distribution; approximate variants may change it.	Qualify by algorithm.	Answering unconditional yes or no.
4	What determines speedup? Draft cost, target verification cost, acceptance rate, proposal length, hardware, and serving load.	Cost model.	Using acceptance rate alone.
5	Why discard tokens after first rejection? Later proposals were conditioned on the rejected token and no longer match the committed prefix.	Autoregressive dependency.	Keeping later independently "correct-looking" tokens.
6	How can target verify in parallel? Causal masked forward pass scores all proposed positions together.	Short-prefill view.	Claiming causal dependency disappears mathematically.
7	What is acceptance probability in exact sampling? $\min(1, p(x)/q(x))$ with a residual correction on rejection.	Distribution-preserving rejection sampling.	Accepting only when target argmax matches under stochastic sampling.
8	How choose the draft model? Optimize end-to-end latency: cheap enough, sufficiently aligned, compatible tokenizer/vocabulary.	Agreement-cost balance.	Always choosing the smallest model.
9	What happens to KV caches? Maintain draft/target caches; commit accepted states and roll back/discard rejected suffix states.	Transactional cache handling.	Reusing draft K/V in a different target model.
10	When can it hurt? Low acceptance, expensive draft, high server load/batching interference, short outputs, or inefficient verification kernels.	Workload-dependent answer.	Claiming guaranteed speedup.

7. Deep-Dive Questions

1. **Why does the acceptance rule preserve the target distribution?** The accepted mass for token x is $q(x) \min(1, p(x)/q(x)) = \min(q(x), p(x))$. The rejection path samples the remaining target mass $\max(0, p-q)$, so accepted plus correction mass reconstructs p .
2. **How would you select k adaptively?** Estimate recent per-request acceptance and measured draft/verify times. Increase k when agreement is high and verification amortizes well; decrease it after early rejections or when latency/queue pressure rises.
3. **How does continuous batching complicate speculation?** Requests advance by different accepted counts, verification shapes differ, and a long speculative round increases iteration latency for ordinary decodes. Schedulers may batch similar k , cap verification tokens, or disable speculation under load.
4. **What is tree-based speculation?** Propose multiple candidate branches, pack them into a tree-attention verification pass, and accept a matching path. It raises the chance of useful progress but costs more proposal tokens, metadata, and specialized masking.
5. **Can the target reuse target verification K/V?** Yes for the accepted prefix and correction if computed consistently. States after the rejection are invalid because they were conditioned on an uncommitted path.

8. Comparison Tables

Method	Proposer	Correctness	Main cost
Standard decode	None	Target distribution	One target step/token
Draft-model speculation	Smaller model	Exact with correction	Extra model/cache
Self-speculation	Skipped layers/early exit	Can be exact after full verification	Shared-model scheduling complexity
Multi-token heads	Target-trained heads	Depends on verification rule	Training and head overhead
N-gram proposal	Prompt/history lookup	Exact after verification	Low acceptance outside repetitive text
High acceptance		Low acceptance	
Predictable/code/boilerplate text		High-entropy or distribution-mismatched text	
More tokens per target call		Wasted proposal and verification work	
Larger k may help		Smaller k or no speculation may win	

9. Common Mistakes

- Saying draft tokens bypass the target model.
- Claiming exact sampling is just argmax comparison.
- Ignoring the residual distribution after rejection.
- Reusing draft-model K/V as target-model K/V.
- Keeping candidate states after an earlier rejection.
- Optimizing acceptance rate without measuring draft cost.
- Quoting a universal speedup independent of batch/load/hardware.

10. Edge Cases / Special Cases

- Draft and target need compatible tokenization/vocabulary or a more complex mapping.
- EOS in the candidate sequence must stop acceptance/generation correctly.
- Stop strings can cross boundaries between accepted tokens and a correction.
- $q(x)=0$ needs careful formula handling; a sampled draft token itself has positive draft probability.
- Floating-point probability normalization must avoid negative residuals from rounding.
- With constrained decoding, both proposal and verification must respect the same constraints.
- Very short maximum outputs may end before setup overhead is recovered.

11. How to Explain in Interview

“Speculative decoding lets a cheap draft model propose several tokens, then asks the target to verify them together with causal masking. It accepts a consecutive prefix and corrects the first rejection; exact rejection-sampling rules preserve the target distribution. Speedup depends on accepted tokens per round relative to draft and verification cost, so it is not guaranteed.”

12. Quick Revision Notes

- **Pipeline:** draft $k \rightarrow$ target verifies $\rightarrow$ accept prefix $\rightarrow$ correct rejection.
- **Why faster:** multiple committed tokens per expensive target call.
- **Exact rule:** accept with $\min(1, p/q)$; sample from residual on rejection.
- **Critical metric:** expected committed tokens per round, not just proposed tokens.
- **Cache rule:** commit only states on the accepted/corrected path.
- **Trap:** exact speculation changes execution, not the target distribution.

13. Practice Tasks

1. Simulate exact speculative sampling on two small categorical distributions and compare empirical output with p .
2. Build a latency spreadsheet varying k , acceptance probability, draft cost, and verify cost.
3. Trace cache lengths for a five-token proposal rejected at position three.
4. Implement greedy speculation using two tiny next-token functions.
5. Design a scheduler rule that turns speculation off when verification harms TPOT under high load.

14. Final Cheat Sheet

Item	Remember
Core definition	Cheap proposal plus target parallel verification
Why it matters	Breaks the one-expensive-call-per-token latency pattern
Most asked	Exactness, p/q acceptance, first rejection, cost model, cache rollback
Comparisons	Standard vs draft/self/tree speculation; greedy vs exact sampling
One-line answer	“Speculative decoding advances several tokens per target call when a cheap proposer and the target agree.”

Tensor Parallelism

1. Overview

Definition. Tensor parallelism (TP) splits individual model tensors and their operations across multiple GPUs, so all participating GPUs cooperate on each Transformer layer.

When a layer's weights or required compute do not fit efficiently on one device, matrices can be partitioned by rows or columns. Every GPU performs a shard of the matrix multiplication, then collective communication combines or redistributes partial results. TP is used for low-latency inference of large models within a node connected by NVLink/NVSwitch and sometimes across high-speed network fabrics.

It matters because it reduces per-GPU weight memory and per-device compute, but introduces communication on the critical path of almost every layer. Interviewers ask it to test matrix algebra, collective operations, topology awareness, attention-head sharding, and latency scaling limits.

2. Core Idea

For $Y = XW$, split W by output columns:

```
W = [W0 | W1]
Y = [XW0 | XW1]
GPU0 computes Y0    GPU1 computes Y1
```

Each GPU gets a different slice of the output. This is **column parallelism**. If the next operation can consume those slices independently, no immediate gathering is needed.

Split by input rows instead:

```
W = [W0]
    [W1]      X = [X0 | X1]
Y = X0W0 + X1W1
```

Each GPU computes a partial sum; an all-reduce combines them. This is **row parallelism**.

Analogy. Two people assemble a wide report. For one stage, each writes different columns (column split). For another, each calculates part of every final total, so they must add their partial totals together (row split).

Transformer implementations pair a column-parallel first projection with a row-parallel second projection:

```
replicated X
-> column-parallel QKV or MLP up projection
-> local activation/attention work
-> row-parallel output/down projection
-> all-reduce to replicated result
```

This avoids gathering the large intermediate tensor.

3. Important Subtopics

Column-parallel linear layers

Partition output features. Each rank stores different columns and produces different output features. Bias is partitioned accordingly. It suits QKV and MLP expansion projections whose following operations can remain sharded.

Row-parallel linear layers

Partition input features. Each rank consumes its local activation shard and produces a partial output. An all-reduce (or reduce-scatter in a different layout) forms the result.

Collectives

All-reduce sums partial tensors and gives the result to every rank. All-gather concatenates shards on every rank. Reduce-scatter sums then distributes shards. Their latency/bandwidth and implementation topology dominate TP scalability.

Attention-head and KV-head partitioning

Query heads can usually be divided across ranks. With GQA/MQA, KV heads may be fewer than TP ranks, requiring KV replication or special grouping. Cache size per rank depends on the actual sharding.

Vocabulary parallelism

The large embedding/output matrix can be sharded by vocabulary. Computing global softmax or top-k then needs reductions/exchanges, although optimized distributed top-k can avoid gathering all logits.

Topology and degree

TP communicates frequently, so it is usually kept within a node or fast island. Increasing degree saves memory/compute per GPU but makes local GEMMs smaller and collective overhead more dominant.

4. Real-World Example

An 8-GPU NVLink server hosts a model whose FP16 weights are too large for one GPU. With TP degree 8, each GPU stores roughly one eighth of major linear weights. For every Transformer layer, each rank computes its QKV/MLP shards; row-parallel outputs are all-reduced over NVLink. This can give one request access to all GPUs and reduce latency, but at small batch size the per-rank matrices become narrow and communication can prevent linear speedup.

5. Diagrams / Mental Models

Column parallel
X replicated
|
GPU0: $W_0 \rightarrow Y_0$

Row parallel
 $X = [X_0 \mid X_1]$
| |
GPU0 GPU1

```

GPU1: W1 -> Y1
    |
[Y0 | Y1] sharded

      |      |
      P0     P1
      \     /
      SUM (all-reduce) -> Y

```

Approximate layer latency:

$$T_{\text{layer}}(\text{TP}) \approx T_{\text{compute}} / \text{TP} + T_{\text{collective}}(\text{message}, \text{TP}, \text{topology}) + \text{imbalance}$$

Collective	Input per rank	Output per rank	Common TP use
All-reduce	Full-size partial tensor	Full-size summed tensor	Row-parallel output
All-gather	One shard	Concatenation of all shards	Materialize full activation/logits
Reduce-scatter	Full-size partial tensor	One summed shard	Keep result sharded

6. Common Interview Questions

#	Question and answer	Interviewer expects	Common mistake
1	What is tensor parallelism? Sharding tensors/operations of one layer across devices.	Intra-layer cooperation.	Describing replicas processing different requests.
2	Why use it? Fit a large layer/model and reduce per-GPU compute/latency.	Memory plus compute.	Claiming no communication is needed.
3	Column vs row parallel? Column split creates output-feature shards; row split creates partial sums requiring reduction.	Matrix equations.	Naming splits based on activation rather than weight dimensions without explanation.
4	Why pair column then row splits? Local intermediate shards feed the row-parallel layer, avoiding a large intermediate all-gather.	Communication minimization.	Gathering after every linear operation.
5	What collective is common? All-reduce after row-parallel projections; layouts may use reduce-scatter/all-gather.	Operation semantics.	Using broadcast to sum partial results.
6	Why does TP scale poorly across slow links? Communication sits inside each layer and token step.	Frequency and critical path.	Discussing only total bytes.

#	Question and answer	Interviewer expects	Common mistake
7	How is attention sharded? Divide heads/projection dimensions; each rank computes local heads, then output projection combines.	Head independence and output reduction.	Splitting a head arbitrarily without kernel/model support.
8	How does GQA complicate TP? Fewer KV heads may not divide across ranks, causing replication or constrained TP groups.	KV-head mapping.	Assuming query-head divisibility is sufficient.
9	Does TP give linear speedup? Rarely; collectives, smaller GEMMs, imbalance, and launch overhead limit it.	Scaling equation.	Equating 8 GPUs with 8× lower latency.
10	TP vs data parallelism? TP splits one model execution; DP replicates the model and handles different requests.	Memory/communication/use-case contrast.	Calling both "multi-GPU batching."

7. Deep-Dive Questions

- How can all-reduce be decomposed?** Conceptually as reduce-scatter followed by all-gather. Ring algorithms move bandwidth-efficient chunks; tree algorithms can reduce latency for smaller messages. Libraries select algorithms based on topology and size.
- Can communication overlap compute?** Sometimes: split work into chunks, launch collectives on separate streams, or overlap one layer's communication with independent computation. Transformer dependencies limit full overlap because the next layer often needs the reduced result.
- What happens to normalization and residuals?** Designs often all-reduce to a replicated hidden state before residual/norm so these operations are local and identical. Sequence-parallel variants keep some activations sharded and require corresponding collective/layout changes.
- How does TP affect quantization?** Scales and group boundaries must align with shards; local dimensions must satisfy packed-kernel constraints. Communication is often in FP16/BF16 even when weights are low-bit, so quantization may not reduce collective cost automatically.
- How would you choose TP degree?** Use the minimum degree needed for weight/KV capacity and desired latency within the fastest topology, then benchmark. Too high a degree shrinks GEMM efficiency and raises collective overhead.

8. Comparison Tables

Property	Tensor parallelism	Pipeline parallelism	Data parallelism
Model placement	Each layer sharded	Layers/stages split	Full model replicated
One request uses	All TP ranks per layer	All stages in sequence	One replica/group
Frequent communication	Activations/partial sums each layer	Activations at stage boundaries	Little during inference

Property	Tensor parallelism	Pipeline parallelism	Data parallelism
Latency strength	Can reduce per-layer compute	Often adds stage traversal/bubbles	No single-request speedup
Best topology	Very fast links	Can tolerate slower links better	Independent replicas
Split	Local result	Required combination	
Weight columns/output features	Output shard	Keep sharded or all-gather later	
Weight rows/input features	Partial output sum	All-reduce or reduce-scatter	
Vocabulary rows	Logit shard	Distributed top-k/softmax or gather	

9. Common Mistakes

- Confusing TP with replication/data parallelism.
- Gathering intermediate activations unnecessarily between paired linear layers.
- Assuming all-reduce is a synchronization with no data-transfer cost.
- Selecting TP degree from GPU count rather than topology and divisibility.
- Forgetting GQA KV-head constraints.
- Expecting linear latency scaling at small batches.
- Ignoring collective ordering consistency across ranks, which can deadlock.

10. Edge Cases / Special Cases

- Hidden dimensions, head counts, and quantization groups may not divide by TP degree.
- MQA may replicate its single KV head across ranks.
- Small decode GEMMs can become less efficient after excessive sharding.
- Different rank control flow or collective order causes hangs.
- A slow or failed rank stalls the entire TP group.
- Multi-node TP can be necessary for capacity but pays network latency each layer.
- CUDA Graph capture must coordinate stable collective behavior across ranks.

11. How to Explain in Interview

“Tensor parallelism shards each layer's matrices across GPUs. A column-parallel projection produces feature shards; a following row-parallel projection consumes those shards and all-reduces partial outputs, avoiding an intermediate gather. It reduces per-GPU weights and compute, but frequent collectives make fast interconnects and a modest TP degree essential.”

12. Quick Revision Notes

- **Definition:** intra-layer tensor and compute sharding.
- **Column split:** output features sharded.
- **Row split:** partial output sums; reduce required.
- **Pairing:** column → local nonlinearity/attention → row → all-reduce.
- **Main limit:** communication on every layer/token path.
- **Trap:** more TP ranks can make latency worse.

13. Practice Tasks

1. Split a small matrix multiplication by columns and rows and verify reconstructed output.
2. Calculate parameter bytes per rank for TP degrees 1, 2, 4, and 8.
3. Draw collectives for an MLP up/gate/down projection.
4. Estimate compute and ring all-reduce time using link bandwidth and latency.
5. Explain how 8 query heads and 2 KV heads could map to TP degrees 2, 4, and 8.

14. Final Cheat Sheet

Item	Remember
Core definition	Split individual layer tensors and operations across GPUs
Why it matters	Fit large models and share one request's compute
Most asked	Row vs column split, all-reduce, head sharding, scaling limits
Comparisons	TP vs PP vs DP; all-reduce vs all-gather vs reduce-scatter
One-line answer	"Tensor parallelism trades frequent intra-layer communication for lower per-GPU model memory and compute."

Pipeline Parallelism

1. Overview

Definition. Pipeline parallelism (PP) partitions a model's ordered layers into stages placed on different devices. Activations flow from one stage to the next like items through an assembly line.

For inference, PP is primarily a model-capacity technique: each GPU stores only its assigned layers. Multiple microbatches or requests can be in different stages simultaneously to improve throughput. It is used for very large models spanning GPUs or nodes, often combined with tensor and data parallelism.

It matters because it needs less frequent communication than tensor parallelism, but a single token must still traverse all stages and pipeline bubbles can leave devices idle. Interviewers ask about stage partitioning, microbatch scheduling, bubbles, latency, communication, KV-cache placement, and failure synchronization.

2. Core Idea

Split a 12-layer model across three GPUs:

```
GPU0: layers 0-3  -> GPU1: layers 4-7  -> GPU2: layers 8-11
```

Analogy. In a factory, one station builds the frame, another adds electronics, and a third tests the product. One product takes the sum of all station times, but once the line fills, several products are processed at once.

Without microbatches:

```
time 1: GPU0 handles A; GPU1 idle;      GPU2 idle
time 2: GPU0 idle;      GPU1 handles A; GPU2 idle
time 3: GPU0 idle;      GPU1 idle;      GPU2 handles A
```

With multiple microbatches:

```
time 1: GPU0 A
time 2: GPU0 B | GPU1 A
time 3: GPU0 C | GPU1 B | GPU2 A
time 4: GPU0 D | GPU1 C | GPU2 B
```

Step by step:

1. Partition consecutive model layers into stages.
2. Place weights and the corresponding per-layer KV cache on each stage's device.
3. Divide serving work into microbatches/requests.
4. Stage 0 computes and sends boundary activations to stage 1.
5. Later stages run as soon as their input arrives.
6. The last stage computes logits/sampling; the selected token/state needed for the next decode iteration is coordinated back to the pipeline entrance.

3. Important Subtopics

Stage partitioning

Equal layer counts do not guarantee balanced stages: embeddings, attention variants, MoE layers, output heads, and device speeds differ. The slowest stage sets steady-state throughput. Partition by measured latency and memory, not just counts.

Microbatching

Microbatches let stages overlap. More microbatches reduce the relative fill/drain bubble but add scheduling, buffering, and sometimes latency. In online serving, independent requests naturally act as microbatches.

Pipeline bubble

During fill and drain, some stages are idle. For P balanced stages and M equal microbatches in a simple forward pipeline, idealized utilization is approximately:

$$M / (M + P - 1)$$

This model ignores communication and imbalance but explains why small M uses stages poorly.

Boundary communication

Only activations at stage boundaries are sent, usually less frequently than TP collectives. This makes PP more suitable across nodes, though link bandwidth and activation size still matter.

Autoregressive decode scheduling

Each token for a sequence must traverse every stage before the next token is known. Batching many sequences keeps the pipeline full, but single-request latency usually includes all stage times plus transfers.

KV-cache locality

Each stage stores K/V only for its local attention layers. Cache allocation/admission must succeed consistently across stages; one stage running out of blocks limits the whole pipeline.

4. Real-World Example

A model is too large for one 8-GPU node. It is split into four pipeline stages across four nodes, with two-way tensor parallelism inside each node. Activations cross the network only three times per forward traversal, while high-frequency TP all-reduces stay within fast NVLink pairs. Many continuously batched sequences occupy the stages; monitoring uses the slowest-stage queue to rebalance layer assignment.

5. Diagrams / Mental Models

```
requests/microbatches --> [Stage 0] ==activations==> [Stage 1] ==> [Stage 2] --> logits
```

	layers 0-7	8-15	16-23
	KV 0-7	KV 8-15	KV 16-23
Steady state:	C on S0	B on S1	A on S2

Source of idle time	Cause	Mitigation
Fill/drain bubble	Not enough in-flight microbatches	More microbatches/requests
Stage imbalance	One stage takes longer	Repartition layers or heterogeneity-aware placement
Communication stall	Activation transfer not ready	Faster links, overlap, smaller boundary tensors
Decode dependency	Next token waits for logits	Batch independent sequences; cannot remove causality

6. Common Interview Questions

#	Question and answer	Interviewer expects	Common mistake
1	What is pipeline parallelism? Place consecutive layer groups on different devices and pass activations between them.	Inter-layer partition.	Splitting each matrix across GPUs.
2	Why use it? Fit models by distributing layers and improve throughput with overlapped microbatches.	Capacity first, overlap second.	Claiming it always lowers one-request latency.
3	What is a pipeline bubble? Stage idle time during fill/drain or imbalance.	Timeline and utilization.	Calling communication itself the bubble.
4	How do microbatches help? They put different work items on different stages concurrently.	Assembly-line overlap.	Saying a single microbatch uses all stages simultaneously.
5	What determines throughput? The slowest stage in steady state, plus communication/scheduling.	Bottleneck stage.	Averaging stage times.
6	How should layers be partitioned? Balance measured compute and memory while considering boundary transfer.	Heterogeneous layer cost.	Equal layer count only.
7	Where is the KV cache? Each stage owns cache entries for its local layers.	Distributed cache placement.	Keeping all K/V on the first or last GPU.
8	PP vs TP communication? PP sends activations at stage boundaries; TP performs collectives inside many layers.	Frequency/granularity.	Saying PP has no communication.
9	Why can PP hurt decode latency? Each token serially traverses every stage and incurs boundary transfers.	Causal token path.	Assuming stage overlap accelerates one sequence's dependency chain.
10	How do PP and TP combine? Shard layers across stages, then tensor-shard each stage among a local group.	2D/hybrid parallel layout.	Making all GPUs participate in every collective across all stages.

7. Deep-Dive Questions

- Derive simple pipeline utilization.** M microbatches take roughly $M + P - 1$ stage-time slots in a balanced forward pipeline; each stage performs useful work for M , giving $M / (M + P - 1)$ utilization.
- How does stage imbalance change the formula?** The slowest stage defines the cycle time, while faster stages wait. Total time is closer to fill/drain latency plus $(M - 1) \times \max(\text{stage_time})$, with communication included in effective stage time.
- How is sampling handled?** The final stage produces logits and samples or sends logits/top candidates to a coordinator. The chosen token ID must reach the first stage before the next decode traversal;

sending one token is small but synchronization affects scheduling.

4. **How would you recover from a stage failure?** The whole pipeline group normally fails in-flight requests because no alternate stage can complete them. Production systems restart/reroute to another replica and clean distributed cache state; transparent mid-request recovery is complex.
5. **Why might interleaved/virtual stages help?** Assign multiple separated layer chunks per device so work can be scheduled more finely, reducing bubbles/imbalance. It increases transfers and scheduler complexity and is more prominent in training than simple inference paths.

8. Comparison Tables

Property	Pipeline parallelism	Tensor parallelism
Partition unit	Consecutive layer groups	Matrices/heads within a layer
Communication	Boundary activations	Frequent collectives
Single-request latency	Sum of stage path	Can reduce layer compute but adds collectives
Throughput mechanism	Multiple microbatches in flight	Parallel work within same layer
Link tolerance	Often better across nodes	Prefers fastest links
Main inefficiency	Bubbles/imbalance	Collective overhead/small shards
Microbatch count	Bubble	Buffering/latency
Small	Large relative bubble	Low buffering
Large	Better utilization	More in-flight memory and queueing

9. Common Mistakes

- Confusing stages with tensor shards.
- Claiming every GPU works on the same request at the same instant.
- Ignoring fill/drain bubbles.
- Partitioning solely by number of layers.
- Forgetting cache memory and admission must be balanced across stages.
- Assuming microbatching removes a single sequence's autoregressive dependency.
- Placing high-frequency TP collectives across slow nodes when a hybrid mapping could keep them local.

10. Edge Cases / Special Cases

- A stage containing embeddings or the LM head may require much more memory than equal layer stages.
- MoE layers have data-dependent cost and can imbalance a stage.
- One stage's KV pool exhaustion blocks new admissions everywhere.
- Very small batches expose almost the full bubble.
- Network jitter on one boundary propagates backpressure through the pipeline.
- Output sampling settings can complicate batching at the last stage.
- Different devices may justify unequal stage partitions.

11. How to Explain in Interview

“Pipeline parallelism assigns consecutive layer groups to different devices and sends activations between them. It primarily makes a model fit; throughput improves when many microbatches occupy different stages. The slowest stage and fill/drain bubbles limit utilization, and a single generated token still traverses every stage, so PP does not inherently reduce one-request latency.”

12. Quick Revision Notes

- **Definition:** inter-layer/stage partitioning.
- **Throughput:** multiple microbatches occupy different stages.
- **Utilization idealization:** $M/(M+P-1)$ for balanced forward stages.
- **Bottleneck:** slowest stage plus boundary transfers.
- **Cache:** local to the stage's layers.
- **Trap:** pipeline overlap helps throughput more than single-sequence latency.

13. Practice Tasks

1. Draw schedules for 2, 3, and 4 stages with different microbatch counts.
2. Calculate ideal utilization using $M/(M+P-1)$ and compare with a discrete simulation.
3. Partition layer latency values into balanced contiguous stages.
4. Add boundary transfer time and identify the bottleneck stage.
5. Design a hybrid topology with PP across nodes and TP within each node.

14. Final Cheat Sheet

Item	Remember
Core definition	Split ordered layers into stages on different devices
Why it matters	Fits very large models; pipelines independent microbatches
Most asked	Bubble, microbatch, balance, PP vs TP, KV placement
Comparisons	Inter-layer PP vs intra-layer TP; latency vs throughput
One-line answer	“Pipeline parallelism trades stage-boundary latency and bubbles for distributed model capacity and overlapped request throughput.”

Data Parallelism

1. Overview

Definition. In inference data parallelism, multiple replicas of the model process different requests or batches independently. A router distributes traffic among replicas.

Unlike training data parallelism, inference replicas generally do not synchronize gradients or model parameters on every step. The same frozen weights are loaded in each replica, while request state and KV caches remain local. A “replica” may itself be a multi-GPU tensor/pipeline-parallel group.

Data parallelism is the standard way to scale aggregate serving throughput, availability, and geographic reach. Interviewers ask about it to test the distinction between training and inference, routing, load balancing for variable-length stateful jobs, replica memory cost, and how DP composes with model parallelism.

2. Core Idea

Analogy. A store opens four identical checkout counters. Each customer uses one counter; four counters increase total customers served but do not make one checkout four times faster.

```
incoming requests  +--> replica 0: request A, D
                   --+--> replica 1: request B
                   +--> replica 2: request C, E
```

Step by step:

1. Load the same model version on each replica.
2. A router observes replica health, queue, cache capacity, and sometimes prefix locality.
3. Assign each new request/session to one replica.
4. That replica performs prefill and decode, maintaining its local KV cache.
5. Stream the output through the router or directly to the client.
6. Scale replicas based on queueing/throughput and drain them safely during updates.

If the model requires $TP=4$ and there are 16 GPUs, the deployment may run four data-parallel replicas, each containing a 4-GPU TP group.

3. Important Subtopics

Inference versus training DP

Training DP splits examples and all-reduces gradients to keep replicas synchronized. Inference DP serves independent requests with immutable weights and normally needs no per-token collective between replicas.

Load balancing

Round-robin ignores different prompt/output lengths. Least-queued-work, token-based load estimates, KV-block availability, or power-of-two choices better handle heterogeneous LLM requests.

Session affinity

Follow-up turns should often route to the replica holding their KV/prefix cache. Affinity reduces repeated prefill but can cause imbalance. Routers weigh cache locality against queue delay.

Replica granularity

A replica can be one GPU for a small model or a $TP \times PP$ group for a large model. "Data-parallel degree" counts independent model groups, not necessarily individual GPUs.

Weight memory and loading

Every replica stores a complete logical model, so DP multiplies weight memory across the cluster. Memory sharing may exist within a host/process arrangement, but independent GPUs need their own weight shards.

Fault tolerance and rolling updates

Because replicas are independent, traffic can avoid failed or draining replicas. Updates require version-aware routing so a session/cache does not silently cross incompatible model versions.

4. Real-World Example

A service has 32 GPUs. Its model needs 4-way tensor parallelism, so the system creates eight independent TP groups. The load balancer routes new conversations to the group with the lowest estimated remaining token work and enough KV blocks. Later turns prefer the same group for prefix locality unless its queue delay exceeds a threshold. During a model rollout, old and new groups use different routing pools and caches.

5. Diagrams / Mental Models

```

DP replica 0 = [TP ranks 0..3]
client -> request router -> DP replica 1 = [TP ranks 4..7]
DP replica 2 = [TP ranks 8..11]

Within replica: GPUs cooperate on one model execution.
Across replicas: requests are independent; no per-token collective.

```

Router signal	Why it matters
Active sequences	Rough concurrency measure
Queued/scheduled tokens	Better work estimate than request count
Free KV blocks	Admission feasibility
Cached prefix/session	Avoid repeated prefill
Health/version	Correctness and availability

6. Common Interview Questions

#	Question and answer	Interviewer expects	Common mistake
1	What is data parallel inference? Replicated model groups serve different requests independently.	Replica-level concurrency.	Describing tensor sharding.
2	Does DP reduce one request's latency? Generally no; it raises aggregate capacity and can reduce queueing.	Throughput versus service time.	Claiming N replicas make one forward pass N× faster.
3	How does inference DP differ from training DP? No gradient all-reduce per step; weights	Communication distinction.	Saying replicas must all-reduce logits.

#	Question and answer	Interviewer expects	Common mistake
	are frozen and requests independent.		
4	What is a replica for a huge model? A complete logical model implemented by a TP/PP group.	Composition of parallel dimensions.	Equating one GPU with one replica.
5	Why is round-robin weak for LLMs? Requests have variable prompt/output lengths and cache use.	Work-aware routing.	Counting requests as equal jobs.
6	What is session affinity? Route subsequent turns to the cache-owning replica.	Locality benefit and imbalance risk.	Making affinity absolute even when replica is overloaded.
7	What limits DP scale? Traffic, duplicate weight memory, loading time, router/host bottlenecks, and uneven work.	End-to-end constraints.	Saying network collectives dominate between replicas.
8	How do you autoscale? Use queue delay, token backlog, utilization, cache pressure, and startup time.	SLO-aware multi-signal scaling.	GPU utilization alone.
9	How does failure affect requests? In-flight state on that replica is usually lost; new requests route elsewhere and clients retry as allowed.	State-locality consequence.	Assuming another replica has the KV cache automatically.
10	DP vs continuous batching? DP chooses a replica; continuous batching schedules requests within that replica.	Different layers of the system.	Treating them as alternatives.

7. Deep-Dive Questions

1. **How would you route with prefix caching?** Estimate **queue delay + unmatched prefill cost + decode cost** per eligible replica. A cache hit is valuable only if it outweighs waiting behind a busy queue.
2. **How do you avoid load oscillation?** Use slightly stale but smoothed metrics, power-of-two choices, randomized ties, and admission reservations. Sending every new request to the momentarily emptiest replica can create herd behavior.
3. **What consistency is needed during updates?** Pin a request/session to a model and adapter version. New traffic can shift gradually, but cached KV state cannot cross versions unless proven compatible.
4. **Can replicas share weights in host memory?** Processes may map a common host file/page cache, reducing CPU duplication, but GPU device memory still holds a copy of each replica's shards unless special hardware memory sharing is used.
5. **How does DP affect tail latency?** More replicas reduce queueing only with good routing. Stragglers, skewed long jobs, cold replicas, cache affinity, and synchronized batching windows can still create high p99.

8. Comparison Tables

Property	Data parallelism	Tensor parallelism	Pipeline parallelism
Logical model copies	Multiple	One per TP group	One split across stages
Requests	Different per replica	Same request cooperatively	Same request traverses stages
Weight memory cluster-wide	Replicated	Sharded within group	Sharded by layers
Critical communication	Routing/output, little between replicas	Per-layer collectives	Stage activations
Main goal	Aggregate throughput/availability	Fit and accelerate layers	Fit model and pipeline throughput
Routing	Benefit	Drawback	
Round-robin	Simple	Ignores work and memory	
Least active requests	Better than round-robin	Requests have unequal lengths	
Least token backlog	Work-aware	Requires current estimates	
Cache-affinity	Saves prefill	Can create hotspots	

9. Common Mistakes

- Importing training DP's gradient synchronization into inference explanations.
- Counting GPUs rather than logical replicas.
- Assuming each request has equal cost.
- Ignoring local KV state when rerouting sessions.
- Using GPU utilization as the only autoscaling signal.
- Mixing model versions in one cache namespace.
- Claiming DP helps a single request's raw compute latency.

10. Edge Cases / Special Cases

- Low traffic leaves replicas underutilized; consolidation may be cheaper.
- Large-model replica startup can take minutes and affects autoscaling usefulness.
- A nearly finished long generation should not be migrated casually because its KV cache is large.
- Multi-tenant quotas may override globally optimal load balancing.
- Different GPU types require capacity-aware routing and possibly different batch policies.
- Request retries can duplicate non-idempotent side effects around tool calls unless higher layers handle idempotency.
- Prefix-cache affinity can conflict with geographic or compliance routing.

11. How to Explain in Interview

"Inference data parallelism runs multiple independent copies of a logical model and routes different requests to them. It scales aggregate throughput and availability rather than accelerating one forward pass. For LLMs,

routing should consider queued tokens, KV capacity, model version, and session/prefix locality; each replica may itself be a tensor- or pipeline-parallel GPU group.”

12. Quick Revision Notes

- **Definition:** independent replicated model-serving groups.
- **Benefit:** throughput, lower queueing, failure isolation.
- **No inference gradient sync:** replicas use frozen weights.
- **Routing:** token work and cache capacity beat request count.
- **Composition:** total GPUs often $DP \times TP \times PP$ (and possibly EP).
- **Trap:** one replica may contain multiple GPUs.

13. Practice Tasks

1. Given 32 GPUs and $TP=4$, calculate DP degree and route a sample workload.
2. Simulate round-robin versus least-token-backlog routing for variable jobs.
3. Add session affinity and measure saved prefill versus queue imbalance.
4. Design a rolling model update that keeps cache versions correct.
5. Propose autoscaling signals for a service with a five-minute replica startup.

14. Final Cheat Sheet

Item	Remember
Core definition	Replicate the logical model and split requests across replicas
Why it matters	Scale total throughput and isolate failures
Most asked	Training vs inference DP, routing, affinity, replica groups, autoscaling
Comparisons	DP vs TP/PP; round-robin vs work-aware routing
One-line answer	“Data parallel inference scales requests, not one request, by serving them on independent model replicas.”

Expert Parallelism

1. Overview

Definition. Expert parallelism (EP) distributes the experts of a mixture-of-experts layer across devices. Tokens are routed to the devices owning their selected experts, processed there, and sent back to their original sequence positions.

An MoE model may have dozens or hundreds of feed-forward expert networks, but each token activates only a small top- k subset. Storing every expert on every GPU wastes memory; EP shards experts while preserving sparse activation. It is used in large sparse LLM inference and training, commonly combined with data, tensor, and pipeline parallelism.

It matters because it makes a huge expert parameter set fit, but replaces dense local MLP work with data-dependent routing and all-to-all communication. Interviewers ask about dispatch/combine collectives, load imbalance, capacity, token permutation, topology, and the difference between expert parallelism and MoE as a model architecture.

2. Core Idea

Suppose an MoE layer has four experts distributed over two GPUs:

```
GPU0 owns E0, E1
GPU1 owns E2, E3

token A -> E0    token B -> E3
token C -> E2    token D -> E1
```

Before the MoE layer, tokens may be spread by their original batch/sequence placement. The router selects experts; an **all-to-all dispatch** sends each token hidden state to the appropriate owner. Owners group tokens per expert, run expert MLPs, and an **all-to-all combine** returns weighted outputs to original positions.

Analogy. A hospital's reception desk assigns patients to specialist departments. Patients travel to the department, specialists process grouped cases, and reports return to the original care team. If everyone needs cardiology, that department becomes the bottleneck even while others sit idle.

Step by step:

1. Compute router logits for each token.
2. Select top-*k* expert IDs and gating weights.
3. Count tokens per destination and compute permutation offsets.
4. Pack hidden states by destination device/expert.
5. Exchange packed buffers with an all-to-all operation.
6. Run local expert GEMMs, often grouped because expert token counts differ.
7. Exchange results back.
8. Unpermute, weight, and sum expert outputs for each original token.

3. Important Subtopics

Token routing and top-k

The router maps each token representation to expert scores. Top-1 routing sends one copy; top-2 sends two and combines them, raising compute/communication for potentially better model quality.

Dispatch and combine

Dispatch permutes and sends tokens to expert owners. Combine reverses the mapping and applies gate weights. Metadata must preserve original token index and selected-expert slot.

All-to-all communication

Unlike all-reduce, each rank sends different data to every other rank. Message sizes are data-dependent and can be uneven. Latency, bandwidth, topology, and synchronization strongly affect inference.

Load imbalance

Routing can send many tokens to a few experts. The slowest owner delays the collective group. Training may use auxiliary balancing losses; inference must execute the learned routing and can mitigate with batching, placement, replication, or capacity policies.

Expert placement and replication

Uniform placement assigns equal expert counts per GPU. Hot experts may be replicated across devices so tokens use a nearby/less-loaded copy, at extra memory and routing complexity.

Grouped GEMM

Each local expert receives a different number of tokens, so launching one tiny GEMM per expert is inefficient. Grouped GEMM kernels execute multiple variable-size expert matrix operations efficiently in one launch/schedule.

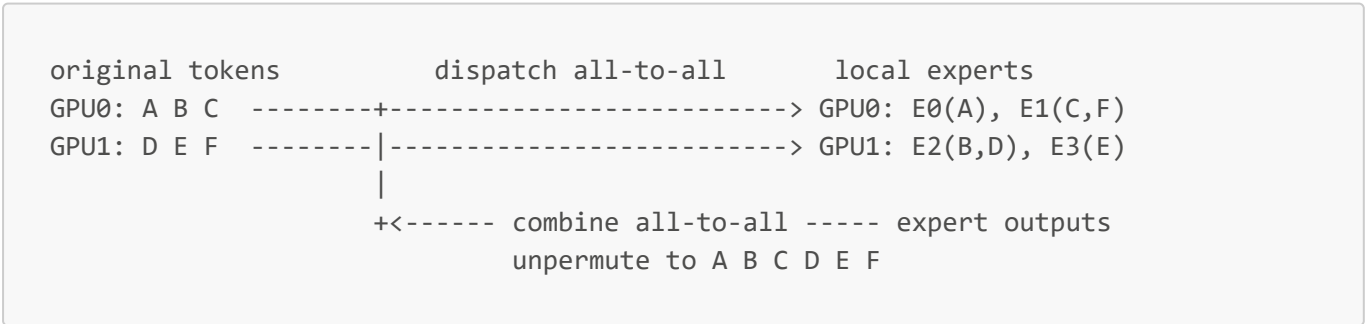
EP group size

The EP group contains devices participating in expert sharding. Larger groups reduce expert memory per GPU but increase all-to-all scope and may cross slower links.

4. Real-World Example

An MoE layer has 64 experts, and each GPU can store eight. Eight GPUs form an EP group. For a prefill batch, the router chooses two experts per token. The runtime packs token states into eight destination buffers, performs all-to-all over NVSwitch, executes local grouped GEMMs for eight experts, then returns and combines outputs. Profiling shows one expert receives 20% of tokens, so its GPU becomes a straggler; the deployment either adjusts batch composition or replicates that hot expert if memory permits.

5. Diagrams / Mental Models



Phase	Data movement	Primary risk
Route	Tokens → expert IDs/weights	Skew or unstable choices
Pack	Original order → expert/destination order	Permutation overhead
Dispatch	Rank-specific all-to-all	Network latency/imbalance

Phase	Data movement	Primary risk
Expert compute	Variable grouped GEMMs	Tiny/uneven matrices
Combine	Outputs back to source ranks	Second communication and bookkeeping

6. Common Interview Questions

#	Question and answer	Interviewer expects	Common mistake
1	What is expert parallelism? Shard experts across devices and route token states to their owners.	Parameter placement plus token movement.	Saying tokens are permanently assigned to one GPU.
2	Why use EP? Fit a large expert set without replicating every expert on every device.	Memory motivation.	Saying it reduces the number of experts.
3	What communication is needed? Usually all-to-all dispatch and all-to-all return/combine.	Two movement phases.	Answering all-reduce only.
4	Why is all-to-all challenging? Each rank sends variable data to many ranks; small/uneven messages and topology create stalls.	Data-dependent communication.	Treating it as a uniform broadcast.
5	What causes load imbalance? Router choices skew tokens toward particular experts/devices.	Token distribution, not only expert count.	Assuming equal experts per GPU means equal load.
6	What is top-k routing's cost? Each token is copied to k experts, increasing expert FLOPs, communication, and combine work.	Quality-cost trade-off.	Saying top-2 costs the same as top-1.
7	Why grouped GEMM? Local experts receive variable small batches; grouping reduces launch overhead and improves device use.	Ragged expert workloads.	Padding every expert to global maximum without considering waste.
8	How can hot experts be handled? Better placement, selective replication, load-aware replica choice, or larger batches; model routing itself may be fixed.	Systems options and constraints.	Silently rerouting a token to a different expert, changing the model.
9	EP vs TP? EP shards independent expert modules and routes tokens; TP shards each matrix and combines partial results.	Sparse routing vs dense cooperation.	Using the terms interchangeably.
10	When does EP scale poorly? Small decode batches, skewed routing, slow interconnects, too-large groups, or tiny per-expert GEMMs.	Communication-to-compute ratio.	Assuming sparsity guarantees speed.

7. Deep-Dive Questions

1. **How does token permutation work?** Build counts for (`destination_rank`, `local_expert`), prefix-sum them into offsets, scatter hidden states plus source metadata into packed order, then use inverse indices to restore outputs. GPU implementations fuse counting/scattering where possible.
2. **What is expert capacity?** Some implementations cap tokens accepted by each expert to a factor times average load. Overflow tokens may be dropped, rerouted, or processed separately. Dropping is especially risky for inference quality, so many inference systems prefer sufficient capacity or exact overflow handling.
3. **How can communication overlap computation?** Split packed tokens into chunks, begin expert compute as destination chunks arrive, or overlap different expert groups. Benefits depend on collective support and enough local work; dependencies prevent arbitrary overlap.
4. **How do TP and EP compose?** Experts can be assigned across EP groups while each expert matrix is itself tensor-sharded. Tokens first route to the expert's group, then TP ranks cooperate on its GEMM. This saves per-rank memory but adds both all-to-all and collectives.
5. **How would topology influence placement?** Keep the highest-volume all-to-all within NVLink/NVSwitch domains when possible, minimize cross-node EP, and place replicated hot experts near demand. Use measured routing matrices rather than assuming uniform traffic.

8. Comparison Tables

Property	Expert parallelism	Tensor parallelism	Data parallelism
What is sharded	Different expert modules	Dimensions of every large tensor	Nothing; full logical model replicated
Token behavior	Routed to selected experts	Same token processed by all TP ranks	Request assigned to one replica
Main collective	All-to-all	All-reduce/all-gather	None per inference step across replicas
Work pattern	Sparse and data-dependent	Dense and regular	Independent workloads
Main imbalance	Router/expert skew	Rank/kernel imbalance	Request-length/queue skew
Top-1 routing		Top-2 routing	
One expert output/token		Weighted combination of two outputs	
Lower compute and traffic		Higher capacity/quality potential	
More sensitive to one choice		More robust expert mixture	

9. Common Mistakes

- Treating EP as synonymous with the MoE architecture.
- Saying sparse activation eliminates communication.
- Assuming equal expert placement ensures equal token load.
- Forgetting the return/combine all-to-all.
- Ignoring permutation and metadata overhead.

- Changing a token's selected expert merely to balance load without accounting for model semantics.
- Using a large cross-node EP group without topology analysis.

10. Edge Cases / Special Cases

- An expert may receive zero tokens; its GEMM must be skipped safely.
- One expert can receive nearly all tokens, making one rank the straggler.
- Top-2 selections can reside on the same or different ranks.
- Padding capacity can waste substantial work during skew.
- Dropped tokens need a defined residual/overflow behavior; silent loss changes the network.
- Decode batches may be too small to fill expert GEMMs, even if prefill performs well.
- Collective participants must agree on send/receive counts to prevent hangs or corruption.

11. How to Explain in Interview

"Expert parallelism places different MoE experts on different GPUs. The router selects top-k experts per token, the runtime permutes and all-to-all dispatches token states to expert owners, local grouped GEMMs run, and results are returned and combined. It saves expert-weight memory per GPU, but load imbalance and data-dependent all-to-all communication are the main scaling limits."

12. Quick Revision Notes

- **Definition:** shard experts; route tokens to owners.
- **Communication:** dispatch all-to-all and return/combine all-to-all.
- **Compute:** variable-size grouped GEMMs.
- **Main bottleneck:** expert skew plus interconnect.
- **Mitigation:** placement, selective replication, batching, careful EP group topology.
- **Trap:** equal numbers of experts do not mean equal load.

13. Practice Tasks

1. Simulate top-2 routing for tokens across four ranks and construct send counts.
2. Implement pack/unpack permutation and verify original output order.
3. Calculate per-rank expert weight memory for different EP degrees.
4. Generate skewed router assignments and measure the slowest-rank load.
5. Design placement for a topology with fast intra-node and slow inter-node links.

14. Final Cheat Sheet

Item	Remember
Core definition	Distribute experts and route token activations to their owners
Why it matters	Fits huge sparse expert parameters across GPUs
Most asked	All-to-all, top-k, imbalance, grouped GEMM, EP vs TP
Comparisons	Sparse EP routing vs dense TP collectives; top-1 vs top-2

Item	Remember
One-line answer	"Expert parallelism saves weight memory by moving tokens to sharded experts, trading local MLP work for all-to-all routing."

MoE Inference

1. Overview

Definition. Mixture-of-experts (MoE) inference executes a sparse neural network in which a router selects a small subset of expert sub-networks for each token, while the rest of the experts remain inactive for that token.

An MoE Transformer usually replaces some dense feed-forward networks with MoE blocks. A model may have a very large total parameter count but activate only a much smaller number of parameters per token. This offers high model capacity at lower FLOPs than activating every parameter, but all expert weights must still be stored somewhere and routing introduces irregularity and communication.

MoE inference is used in large language models that scale parameter capacity without proportional active compute. Interviewers ask about router math, top-k gating, sparse versus total parameters, prefill/decode behavior, memory bandwidth, expert parallelism, imbalance, and why fewer active FLOPs do not automatically mean low latency.

2. Core Idea

For token hidden state x , a router produces expert scores:

```
g = softmax(W_router x)
S = top_k(g)
MoE(x) = sum over i in S of normalized_g_i × Expert_i(x)
```

Often the result is inside the usual residual block, and architectures may also include one always-active shared expert.

Analogy. A general hospital has many specialists. Each patient visits only the one or two relevant specialists, so the hospital contains enormous collective expertise without every specialist examining every patient. But specialists require offices (weight memory), routing takes time, and popular departments form queues.

Small example with four experts and top-2 gating:

```
router scores: E0=0.10, E1=0.55, E2=0.30, E3=0.05
selected: E1 and E2
renormalized weights: 0.647 and 0.353
output = 0.647*E1(x) + 0.353*E2(x)
```

Step by step:

1. Dense attention and normalization produce each token's hidden state.
2. Router projection computes logits over experts.
3. Top- k selection chooses expert IDs and gate weights.
4. Tokens are grouped/routed to the selected experts.
5. Expert MLPs process their assigned token batches.
6. Outputs are weighted, combined, restored to token order, and added through the residual path.
7. Subsequent dense/MoE layers repeat with new routing decisions.

3. Important Subtopics

Total versus active parameters

Total parameters determine storage and model capacity. Active parameters per token determine much of the arithmetic. A 200B-total model activating 20B parameters per token still needs storage for roughly 200B parameters across the deployment.

Router/gating network

The router is usually a learned linear projection followed by top- k selection and normalization. Its FLOPs may be small, but it controls correctness, load distribution, and which weights must be accessed.

Routed and shared experts

Routed experts are sparsely selected. Shared experts run for every token and capture common knowledge, adding dense compute but possibly improving quality and reducing pressure on routed experts.

Top-1 versus top-2 or higher

Higher k activates more expert capacity and may improve quality, but roughly multiplies expert compute, weight reads, and routing traffic. The exact factor depends on shared computation and implementation.

Prefill versus decode

Prefill provides many tokens, producing larger per-expert batches and efficient grouped GEMMs. Decode may have only one token per active sequence; routing fragments a modest batch among many experts, creating tiny GEMMs and making weight access/communication dominant.

Expert weight loading and locality

Even inactive experts occupy memory. If experts are offloaded to CPU or storage, unpredictable per-token choices can cause large transfer latency. GPU-resident experts or carefully designed caching/replication are preferred for strict latency.

Routing imbalance and capacity

Some experts become popular by token/domain/language. Training balancing losses encourage distribution but do not guarantee uniform production traffic. Runtime must handle skew without corrupting model semantics.

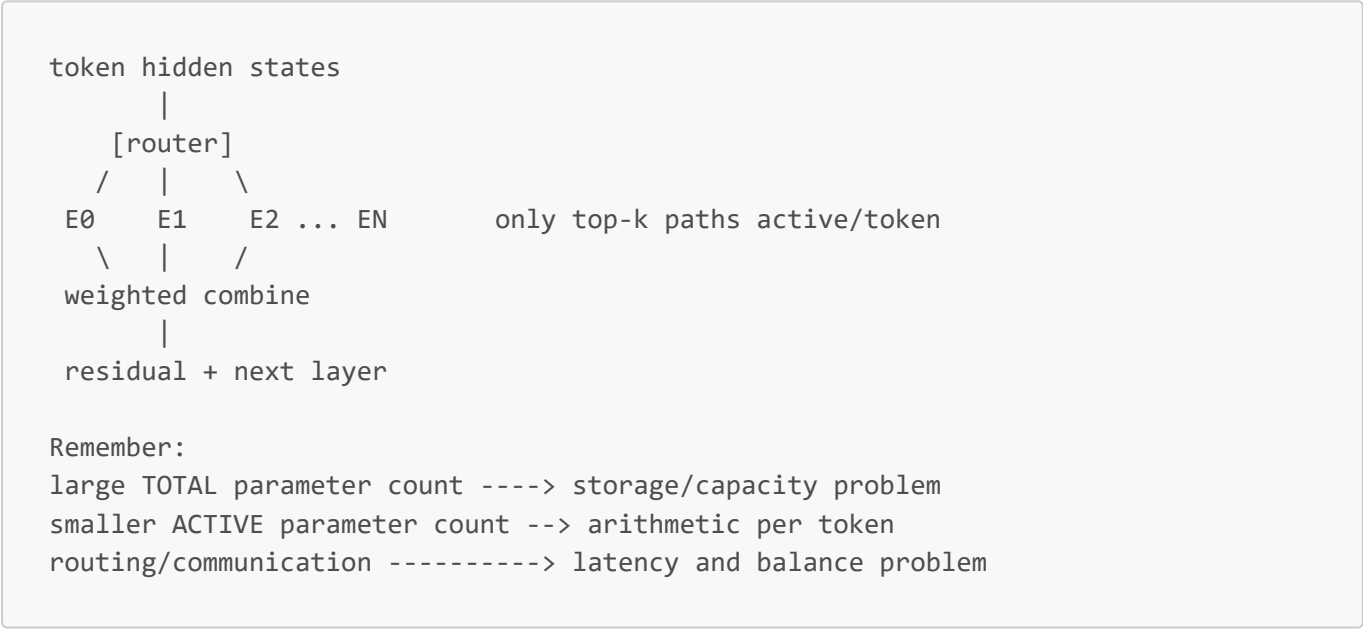
Expert parallel execution

When experts are sharded, MoE inference uses EP dispatch/combine. When all experts fit locally, no network all-to-all is needed, but memory footprint and local weight bandwidth remain large.

4. Real-World Example

A multilingual MoE model has 64 routed experts per MoE layer and selects two per token. English traffic disproportionately activates a subset of experts. During long-prompt prefill, thousands of tokens create healthy expert batches; during interactive decode, 64 active sequences yield only 128 token-expert assignments spread unevenly across 64 experts. The service therefore uses continuous batching, grouped GEMM, expert placement based on routing telemetry, and fast intra-node EP, while tracking both per-expert load and inter-token latency.

5. Diagrams / Mental Models



Dense FFN	MoE FFN
Same MLP weights for every token	Router selects a few expert MLPs per token
Regular GEMMs	Permutation plus variable grouped GEMMs
All FFN parameters active	Small fraction active per token
Easier batching	Expert batches can be tiny/skewed
Weight size tracks active compute	Total storage can far exceed active compute

6. Common Interview Questions

#	Question and answer	Interviewer expects	Common mistake
1	What is an MoE layer? A router chooses a sparse subset of expert networks per token and combines	Router, top-k, experts, weighted	Saying one expert is chosen once for the

#	Question and answer	Interviewer expects	Common mistake
	their outputs.	combine.	whole request.
2	Why use MoE? Increase parameter capacity/quality without activating all parameters for each token.	Total vs active parameters.	Saying unused experts need no storage.
3	What are total and active parameters? Total includes every expert; active includes selected experts plus dense/shared components for a token.	Memory vs FLOPs distinction.	Comparing models only by total parameter count.
4	How does routing work? Learned logits, top-k selection, gate normalization, then expert execution and weighted combination.	Complete data path.	Stopping at top-k without combine.
5	Why can MoE be slow despite low active FLOPs? Irregular routing, small GEMMs, expert weight bandwidth, permutations, all-to-all, and imbalance.	Systems bottlenecks.	Equating fewer FLOPs directly with lower latency.
6	Top-1 vs top-2? Top-2 uses/combines two experts, usually more compute/traffic with potential quality benefits.	Explicit trade-off.	Treating the second expert as a fallback only.
7	Why is decode harder than prefill? Fewer tokens split across many experts produce tiny uneven workloads and poor arithmetic intensity.	Phase-specific batching.	Saying decode activates fewer experts by definition.
8	What is a shared expert? An always-active expert path alongside routed experts.	Dense common path.	Calling a replicated expert automatically "shared."
9	How is MoE distributed? Commonly shard experts with EP; optionally TP-shard each expert and PP-shard layers.	Hybrid parallelism.	Assuming expert parallelism alone covers all model tensors.
10	How do you measure MoE serving health? TTFT/TPOT, tokens per expert, imbalance/straggler ratio, all-to-all time, grouped-GEMM efficiency, memory, and quality.	Model plus systems metrics.	Reporting average GPU utilization alone.

7. Deep-Dive Questions

- 1. What is capacity factor and why is dropping problematic in inference?** Capacity factor sets per-expert slots relative to average expected load. Dropping overflow may stabilize training throughput but changes token outputs at inference; exact serving should provision, reroute only as defined by the model, or process overflow through a correct fallback.
- 2. Why can MoE have lower FLOPs but higher memory pressure than a dense model?** Only selected experts compute, but all weights must be resident/distributed. Decode may read selected expert weights for very few tokens, giving little reuse, while the total weight set consumes capacity.

3. **How would you optimize decode MoE?** Increase useful token batching through continuous batching, use fused route/permute kernels, grouped GEMM, topology-aware EP, hot-expert replication where semantics permit identical copies, and quantized expert weights. Validate TPOT under realistic routing skew.
4. **Can tokens be rerouted for load balance?** Only if the model/algorithm defines an equivalent choice or a capacity fallback. Arbitrarily replacing a selected expert changes the function. Routing to an identical replica of the same expert is safe if weights/configuration match.
5. **How does quantization interact with MoE?** Quantizing expert weights reduces the large total memory footprint and bandwidth. However, per-expert/group scales, small irregular GEMMs, packing divisibility, and kernel availability matter; dense attention/router/shared parts may use different precision.

8. Comparison Tables

Property	Dense Transformer	MoE Transformer
FFN choice/token	One fixed FFN	Top-k routed experts, sometimes shared expert
Total vs active params	Nearly the same per layer	Total can be much larger than active
Compute regularity	High	Data-dependent
Communication when sharded	TP collectives	Often EP all-to-all plus possible TP
Primary serving issue	Weight/KV bandwidth	Weight capacity, routing, tiny GEMMs, imbalance
Concept	Meaning	Do not confuse with
MoE	Sparse model architecture	A distribution strategy
Expert	Usually an FFN subnetwork	A whole independent LLM
Router	Chooses expert IDs/weights per token	Request load balancer
Expert parallelism	Places experts across devices	The MoE computation itself
Data parallelism	Replicates logical MoE model groups	Replicating a hot expert within an EP group

9. Common Mistakes

- Assuming active parameter count is the deployment memory requirement.
- Saying one request or sequence chooses one expert permanently.
- Ignoring the weighted combine after expert execution.
- Treating lower FLOPs as guaranteed lower latency.
- Assuming training-time balance guarantees production balance.
- Confusing a shared expert with a hot expert replica.
- Discussing expert compute without token permutation and communication.

10. Edge Cases / Special Cases

- No tokens may select an expert in a given iteration.
- All tokens may concentrate on one expert or one device.
- Ties and numerical differences in top-k can change routing; deterministic requirements need careful kernels.
- Top-k normalization conventions vary by architecture; do not assume weights sum to one before/after selection without checking.
- Shared experts add active parameters to every token and must be included in FLOP estimates.
- Offloaded experts can create unpredictable latency from cold loads.
- Speculative decoding must verify routing decisions through the target MoE model; a draft's expert choices are not reusable target states.

11. How to Explain in Interview

"MoE inference replaces selected dense FFNs with many experts and a router that activates only top-k experts per token. This gives a large total parameter capacity with much lower active compute, but the deployment must still store all experts and efficiently permute, batch, and sometimes all-to-all tokens. Decode is challenging because small token batches fragment into tiny, imbalanced expert GEMMs."

12. Quick Revision Notes

- **Formula:** $\text{output} = \sum \text{gate}_i \times \text{Expert}_i(x)$ for selected top-k experts.
- **Total parameters:** storage/capacity; **active parameters:** per-token arithmetic.
- **Prefill:** many tokens $\rightarrow$ healthier expert batches.
- **Decode:** few tokens spread across experts $\rightarrow$ tiny/skewed GEMMs.
- **Distribution:** EP shards experts; TP can shard inside experts; DP replicates groups.
- **Trap:** sparse compute does not mean sparse storage or automatically low latency.

13. Practice Tasks

1. Implement a toy top-2 MoE layer in Python with route, group, expert MLP, and combine steps.
2. Given total/dense/shared/expert parameter counts, calculate active parameters for top-1 and top-2.
3. Compare expert batch-size distributions for 4,096-token prefill and 64-sequence decode.
4. Profile a naive per-expert loop versus grouped or batched matrix operations.
5. Design a deployment using DP, PP, TP, and EP for a model and justify which communication stays within a node.

14. Final Cheat Sheet

Item	Remember
Core definition	Router activates and combines a sparse top-k subset of expert FFNs per token
Why it matters	Large model capacity without dense activation of every parameter
Most asked	Total vs active params, routing, top-k, decode inefficiency, EP
Comparisons	Dense vs MoE; top-1 vs top-2; MoE architecture vs EP placement

Item	Remember
One-line answer	"MoE inference saves arithmetic through sparse expert activation, but pays in weight storage, irregular batching, routing, and communication."